Tunisian Republic
Ministry of Higher Education and
Scientific Research

University of Sfax
National Engineering School of Sfax



Major: Computer Engineering

End of Year Project

Presented in :

National Engineering School of Sfax
Department of Computer Engineering and Applied Mathematics

Subject :

A Multi-Agent Approach for End-to-End BDD Test Automation

Reported by :

Maryem KBAIER

Imen KAMMOUN

Emna KAANICHE

Academic examiner : Afef MDHAFFAR
Academic supervisor : Mariam LAHAMI

Academic Year: 2025/2026

Acknowledgements

This work was made possible through the support and contributions of several individuals and institutions, to whom we express our sincere and profound gratitude.

We would like to extend our deepest appreciation to Mrs. **Mariam LAHAMI**, our supervisor at the National School of Engineers of Sfax, for her unwavering guidance, insightful feedback, and continuous support throughout the course of this project. Her expertise and commitment were instrumental in shaping the direction and ensuring the successful completion of this work.

We also wish to acknowledge all those who have contributed, directly or indirectly, to the realization of this project. Their support and encouragement have been greatly valued.

Contents

General Introduction **1**

1 Project Context and Foundations **2**

1.1 Introduction 2

1.2 Background materials 2

1.2.1 Microservices architecture 2

1.2.2 Software testing paradigms 4

1.2.3 Large Language Models and Transformer Architecture 5

1.2.4 Prompt Engineering Strategies 6

1.2.5 Retrieval-Augmented Generation (RAG) 9

1.2.6 Multi-Agent Systems (MAS) 10

1.3 Related Work 12

1.3.1 Comprehensive Evaluation and Insights Into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation 12

1.3.2 Increasing Test Coverage by Automating BDD Tests in Proofs of Concepts (POCs) using LLMs 13

1.3.3 Unit Test Generation Multi-Agent AI System for Enhancing Software Documentation and Code Coverage 13

1.3.4 Discussion 14

1.4 Problem Statement 14

1.5 Proposed Solution: VORTEX - A Multi-Agent BDD End To End System Approach to Software Test Automation 15

1.5.1 Core Concept and Philosophy 15

1.6 Functional and Non-Functional Requirements 15

1.6.1 Functional Requirements 15

1.6.2 Non-Functional Requirements and Their Realization in the Proposed Approach . . . 16
1.7 Conclusion . . . 17

2 System Architecture and Design 18
2.1 Introduction . . . 18
2.2 The System Under Test Overview . . . 18
2.2.1 Functional Analysis and Requirements . . . 19
2.2.2 Static Design and Domain Model . . . 22
2.3 Multi-agent-based Test System . . . 23
2.3.1 Scenario Designer Agent . . . 23
2.3.2 Gherkin Generator Agent . . . 25
2.3.3 Gherkin Validator Agent . . . 27
2.3.4 Test Writer Agent . . . 28
2.3.5 Test Executor Agent . . . 30
2.3.6 Failure Analyst Agent . . . 30
2.3.7 Coverage Analyst Agent . . . 31
2.3.8 Retrieval-Augmented Generation (RAG) . . . 31
2.4 Conclusion . . . 33

3 Implementation and Experimental Results 34
3.1 Introduction . . . 34
3.2 Prototype and Architecture . . . 34
3.2.1 System Pipeline: Architecture Overview . . . 34
3.2.2 Prototype Implementation and Tooling . . . 35
3.2.3 RAG Integration . . . 38
3.3 Graphical User Interface (GUI) . . . 42
3.4 Experimental Setup and Evaluation . . . 47
3.4.1 Per-Agent LLM Selection . . . 48
3.4.2 Performance Analysis Across Versions . . . 50
3.4.3 Comparative Analysis: Expert vs. LLM-Generated Scenarios . . . 54
3.5 Conclusion . . . 57

General Conclusion 58

List of Figures

1.1 Logical overview of the microservice-based architecture. 3
1.2 An example application for RAG on a PC, demonstrating the difference between a standalone base model and a RAG-enhanced model[1]. 10
1.3 Architecture Overview of the Multi-Agent System[2]. 11

2.1 Use Case Diagram of the Leave Request Management System, distributed across User and Leave Management microservices. 19
2.2 Class Diagram illustrating the domain model of the Leave Request Management System. 22
2.3 Overview of the multi-agent-based test system for automated BDD test generation. 23
2.4 Retrieval-Augmented Generation (RAG) Pipeline for Enhanced Artifact Generation. 32

3.1 Architecture and communication flows 35
3.2 RAG Integration Overview 39
3.3 VORTEX dashboard with coverage orchestration and agent workflow tracker. 43
3.4 Microservices configuration and run settings panel. 43
3.5 Requirements input with user stories, business rules, and execution results panel. 44
3.6 Agent progression workflow with completed, active, and pending steps during test execution. 45
3.7 Pipeline console showing real-time execution logs from the test executor agent. 45
3.8 Execution results with coverage metrics, quality gate, and generated artifacts. 46
3.9 Cucumber HTML report showing passed and failed test scenarios. 46

3.10 Detailed error view with assertion failures and stack traces for security scenarios. 47
3.11 Gherkin GeneratorAgent quality benchmark across three LLM candidates. 49
3.12 Gherkin Generator Agent token usage benchmark. 50
3.13 Evaluation metrics and scenario count: One Loop vs. Without Loop. . . . 51
3.14 JaCoCo code coverage comparison: Without Loop vs. One Loop iteration. 53

List of Tables

1.1 Summary of research approaches for test generation 14

3.1 RAG Knowledge Base Datasets 39
3.2 RAG Retrieval Configuration 40
3.3 Evaluation metrics and their role in the pipeline funnel. 48
3.4 Numerical results extracted from the quality benchmark run. 49
3.5 Numerical comparison of pipeline variants with and without the feedback loop. 51
3.6 Performance comparison across all three pipeline versions. 52
3.7 JaCoCo coverage metrics extracted from pipeline execution logs. 53

General Introduction

Today, the quality and reliability of software systems represent a major strategic focus. However, development teams face complex challenges in validating these increasingly intricate applications. Writing and maintaining functional tests require a considerable and repetitive manual effort. These difficulties often result in time constraints, slowed delivery cycles, and incomplete testing coverage, struggling to encompass both business expectations and technical edge cases.

To overcome these obstacles, it has become necessary to modernize and automate quality control processes. The proposed solution is the design of an innovative tool exploiting the advanced capabilities of Artificial Intelligence. The objective is to fully automate the generation of clear and directly usable test scenarios. By adopting a hybrid approach, the tool first relies on business requirements to create tests aligned with user expectations, then automatically enriches them to ensure technical completeness and comprehensive coverage.

This report details the reflection and development process of this project and is structured into three main chapters. The first chapter establishes the project context and foundations, outlining the theoretical background, the problem statement, and the proposed approach along with the system requirements. The second chapter focuses on the system architecture and design, detailing the global workflow and the prompt-based multi-agent system. Finally, the third chapter presents the technical implementation and the experimental results, including evaluation metrics and a comparative analysis.

In conclusion, this report highlights the contribution of artificial intelligence in software engineering, demonstrating how this project aims to significantly reduce manual effort while ensuring better quality and reliability of the final product.

Chapter 1

Project Context and Foundations

1.1 Introduction

This chapter establishes the foundational framework of the project. Its purpose is to define the environment in which the research is conducted and to clarify the motivations behind the development of an automated testing solution.

The chapter is organized as follows: first, it provides the theoretical background and a review of related work to situate the project within the current state of software engineering. Second, it formalizes the problem statement, highlighting the inefficiencies of manual testing in modern architectures. Finally, it outlines the proposed approach and specifies the functional and non-functional requirements that the system must satisfy. This structure ensures a clear transition from the conceptual challenges to the technical specifications of the solution.

1.2 Background materials

This section reviews the key technologies and theoretical principles that form the basis of our proposed solution. It provides the essential background required to fully understand the system design and implementation presented in subsequent sections.

1.2.1 Microservices architecture

The microservices architecture is an architectural style that organizes an application as a collection of loosely coupled, autonomous services aligned with specific business domains. By departing from traditional monolithic structures, this approach facilitates the independent development, deployment, and scalability of each functional component, thereby enhancing the overall agility and maintainability of the system.[3]

1.2.1.1 Core characteristics

According to Microsoft's architectural guidelines [4], a microservice-based system is defined by the following key traits:

- **Decoupling:** Services are loosely coupled and communicate via well-defined Application Programming Interfaces (APIs) (typically using the REpresentational State Transfer (REST) or messaging queues), allowing them to evolve independently.
- **Single Responsibility:** Each service focuses on a specific business capability (e.g., "Order Management" or "User Authentication").
- **Decentralized Data:** Each microservice manages its own private database to ensure data encapsulation and prevent tight coupling at the persistence layer.
- **Polyglot Programming:** Teams can choose the most appropriate technology stack (language, database, framework) for each specific service.

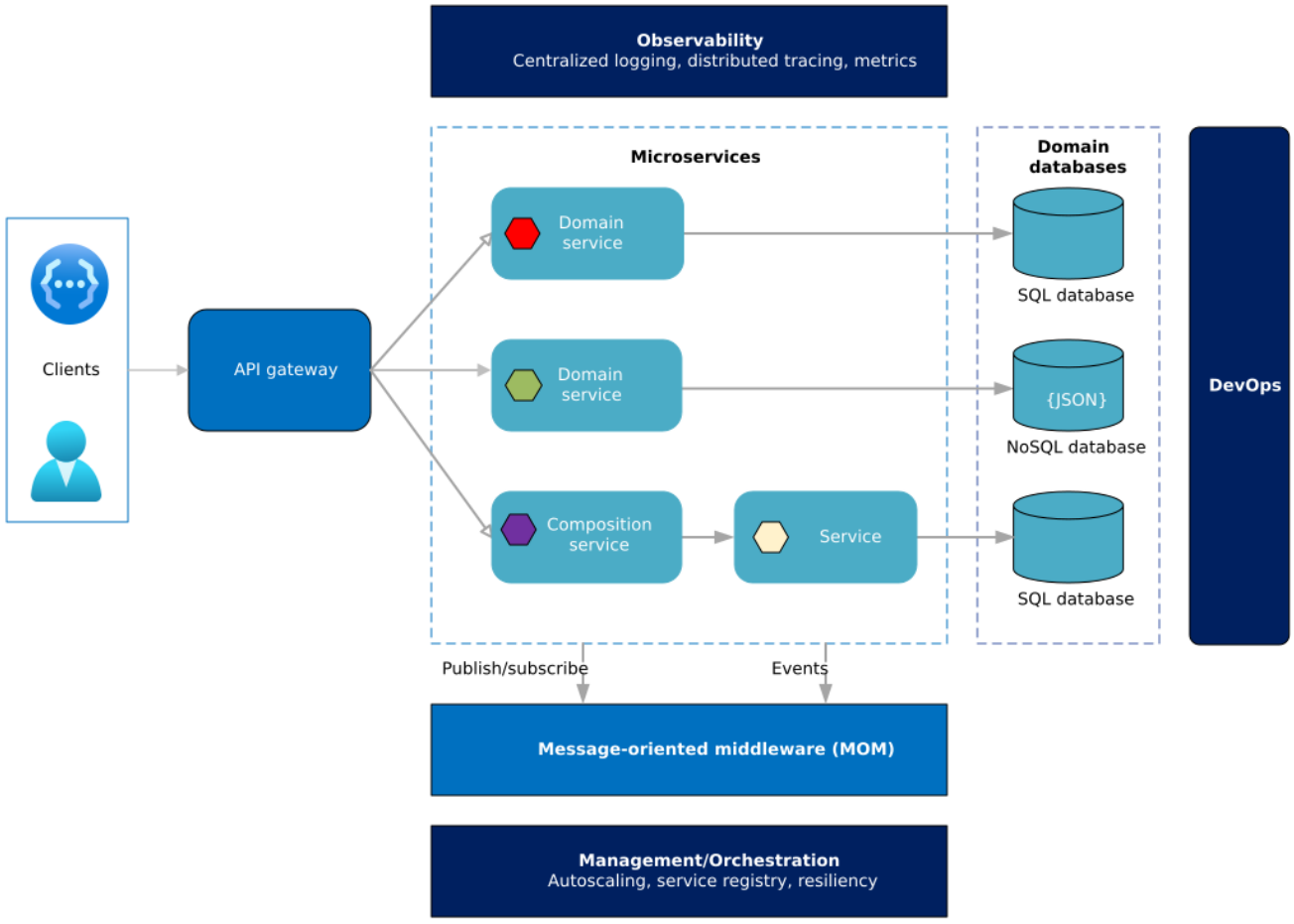


Figure 1.1: Logical overview of the microservice-based architecture.

1.2.1.2 Relevance to automated testing

In microservice-based architectures, the distributed nature of services renders manual functional testing unsustainable and error-prone due to complex inter-service dependencies. Automated, BDD-based testing is therefore critical to preventing regressions across

interconnected business workflows, ensuring system integrity and reliability throughout the development lifecycle.

1.2.2 Software testing paradigms

To ensure the reliability of distributed systems, modern quality assurance relies on specific testing levels and collaborative methodologies. Within the scope of this project, two concepts are fundamental: Behavior-Driven Development (BDD) and End-to-End (E2E) testing.

1.2.2.1 Behavior-Driven Development (BDD)

Behavior-Driven Development (BDD) [5] is an agile software development methodology designed to bridge the communication gap between technical teams (developers, QA) and non-technical stakeholders (business analysts, product owners). Instead of focusing solely on technical implementation, BDD emphasizes the system's expected behavior from the user's perspective. This behavior is documented using a domain-specific, human-readable language called Gherkin, which relies on a strict structural syntax: `Given` (initial context), `When` (action or event), and `Then` (expected outcome).

Here is an example of Gherkin syntax for a purchase scenario:

```
Feature: Product Purchase
Scenario: Successful checkout
Given the user is on the product page for "Laptop"
When the user adds the product to the shopping cart
And the user proceeds to checkout
Then the order should be confirmed successfully
```

Listing 1.1: Example of Gherkin syntax for a purchase scenario.

1.2.2.2 End-to-End (E2E) Testing

While unit tests isolate individual functions, End-to-End (E2E) testing [6] evaluates the entire software product from start to finish to ensure the application flow behaves as expected under real-world scenarios. It validates the integrated system as a whole, encompassing all interconnected components such as user interfaces, backend APIs, databases, and network configurations.

In the context of a microservice-based architecture, E2E testing is inherently complex but absolutely critical. A single business workflow often traverses multiple independent services and their respective databases. E2E testing guarantees that the data integrity and communication protocols between these decoupled services function correctly.

By combining these two paradigms, our project leverages BDD to formalize complex business workflows into Gherkin, which are then executed as automated E2E tests to

validate the complete distributed microservice ecosystem.

1.2.3 Large Language Models and Transformer Architecture

A Large Language Model [7] (LLM) is a type of artificial intelligence that uses machine learning techniques to understand and generate human language. LLMs can be incredibly valuable for companies and organizations looking to automate and enhance various aspects of communication and data processing.

1.2.3.1 The Transformer Architecture

The backbone of modern LLMs is the Transformer architecture [8]. Before Transformers, models processed text sequentially, making it difficult to retain context over long paragraphs. The Transformer solved this through a mechanism called Self-Attention. This allows the model to evaluate the importance of every word in a sequence relative to all other words simultaneously, regardless of their position. This architectural breakthrough is what enables LLMs to deeply understand the context of technical documentation, such as User Stories, and translate them into structured formats like Gherkin.

1.2.3.2 Model Parameters vs. API Inference Parameters

When discussing LLM parameters, a critical distinction must be made between internal model weights and inference settings:

Internal Parameters (Model Weights): These are the billions of neural connections established during the model's training phase (e.g., 175 billion parameters in GPT-3). They define the model's knowledge capacity. In an API-driven architecture like ours, these parameters are abstracted and managed entirely by the cloud provider.

Inference Parameters (API Configurations): These are the dynamic settings passed alongside the prompt during an API call to dictate how the model generates its response. For automated test generation, mastering these parameters is vital.

1.2.3.3 Key inference parameters for automated testing

To ensure the LLM behaves predictably as a technical agent rather than a creative chatbot, specific API parameters must be strictly controlled [9]:

- **Temperature:** This parameter controls the randomness of the output, typically ranging from 0.0 to 2.0. A high temperature makes the model more creative and unpredictable. For our project—where strict adherence to BDD Gherkin syntax and executable code is required—the temperature is explicitly set to 0.0 (or near zero) to ensure deterministic, highly predictable, and reproducible outputs.

- **Top-P (Nucleus Sampling):** Similar to temperature, it controls vocabulary diversity. Lowering this value forces the model to only consider the most probable next words, further reducing the risk of syntax hallucinations.
- **Max Tokens:** This limits the maximum length of the generated response. It acts as a safeguard to prevent infinite loops or excessively long outputs that could crash the execution pipeline.

1.2.4 Prompt Engineering Strategies

Prompt engineering[10] is the systematic process of designing, structuring, and optimizing input queries to effectively guide Large Language Models (LLMs) toward generating accurate and contextually relevant outputs. In the realm of automated software testing, the quality of the prompt directly dictates the reliability and syntactic correctness of the generated code. Two primary techniques are widely utilized: Zero-Shot and Few-Shot prompting.

1.2.4.1 Zero-Shot Prompting

Zero-shot prompting[11] involves presenting a task to the LLM without providing any prior examples or demonstrations within the prompt. The model must rely entirely on its pre-trained knowledge base to infer the rules and generate a response. While highly efficient for general-purpose natural language tasks, zero-shot prompting frequently struggles with domain-specific constraints. In our context, asking an LLM to generate test scripts via zero-shot often leads to structural hallucinations or syntax errors, as the model lacks immediate context regarding the expected output format.

```
Zero-Shot Prompting Example
System Prompt: Translate the following user story into
   Gherkin syntax: "As a bank customer, I want to
   withdraw cash from the ATM so that I can have
   physical money."

LLM Output (Unpredictable & Unstructured):
To test this, the customer needs to insert their card
   into the ATM, type their PIN code, and select the
   withdrawal option. If the account has enough money,
   the machine will give the cash and return the card.

(Note: The model completely missed the strict Given/When
   /Then BDD structure because it was not explicitly
   demonstrated).
```

Listing 1.2: Zero-Shot Prompting Example

1.2.4.2 Few-Shot Prompting

Few-shot prompting[12] mitigates the limitations of zero-shot by embedding a limited number of input-output examples (the "shots") directly within the prompt's context window. This technique leverages the LLM's capacity for *in-context learning*. By demonstrating the exact expected format, constraints, and underlying logic, few-shot prompting significantly enhances the model's accuracy, consistency, and adherence to specific structural guidelines.

```
Few-Shot Prompting Example

System Prompt: Translate user stories into strict
   Gherkin syntax. Use the following structure as a
   template:

Example Input: "As a user, I want to log out so my
   account is secure."

Example Output:
Feature: User Logout
Scenario: Successful logout
Given the user is logged into the application
When the user clicks the "Logout" button
Then the system should redirect them to the login page

Task: Translate this user story: "As a bank customer, I
   want to withdraw cash from the ATM so that I can have
    physical money."

LLM Output:
Feature: ATM Cash Withdrawal
Scenario: Successful cash withdrawal
Given the customer has inserted a valid bank card
And the account balance is sufficient
When the customer requests a cash withdrawal
Then the ATM should dispense the requested physical
   money
```

Listing 1.3: Few-Shot Prompting Example

1.2.4.3 Recommended Prompt Architecture: The CO-STAR Framework

While few-shot prompting establishes a pattern for the model to follow, the reliability of the output is significantly bolstered by adopting a structured prompt architecture. We recommend the **CO-STAR** framework[13], which forces the prompt to contain all necessary dimensions of a high-quality instruction. By adhering to this structure, you minimize ambiguity and ensure that the LLM consistently generates code that complies with project-specific syntax requirements.

The CO-STAR framework consists of the following components:

- **C - Context:** Providing the background information or the scenario that the LLM needs to understand.
- **O - Objective:** Defining exactly what the task is that the LLM needs to perform.
- **S - Style:** Specifying the writing style or the specific coding standard to follow.
- **T - Tone:** Defines the stylistic and emotional quality of the response (e.g., formal, neutral, cautious).
- **A - Audience:** Defining who will read the generated code or documentation.
- **R - Response format:** Explicitly defining the structure, file format, or syntax required for the output.

```
CO-STAR Structured Prompt Example

Context: You are a QA Automation Engineer working on a
   banking application.
Objective: Translate user stories into Gherkin feature
   files.
Style: Use strict BDD syntax with Given/When/Then blocks
   .
Tone: Technical and precise.
Audience: Developers and stakeholders.
Response format: Output only the Gherkin code block.

[...Insert Few-Shot examples here...]

Task: Translate: "As a bank customer, I want to withdraw
    cash from the ATM."
```

Listing 1.4: CO-STAR Structured Prompt Example

By explicitly structuring the prompt using these six pillars, we reduce the "cognitive load" on the LLM, resulting in significantly fewer syntax hallucinations and a more consistent output format suitable for automated pipelines.

1.2.4.4 Application to BDD Automation

In our approach, we utilize few-shot prompting in combination with the CO-STAR prompt architecture, as these techniques significantly enhance the accuracy and consistency of generated outputs, ensuring strict adherence to the required structures.

1.2.5 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a technique that enhances the accuracy and reliability of generative AI models by fetching facts from an external knowledge base. As conceptualized in recent literature and industry standards, relying solely on a base Large Language Model (LLM) is akin to taking a closed-book exam; the model must rely entirely on the static knowledge it acquired during its initial training. RAG transforms this into an open-book exam, allowing the model to browse external, up-to-date, or proprietary data sources to ground its responses in verifiable facts[14].

1.2.5.1 How RAG Works

The RAG architecture effectively bridges the gap between general language understanding and domain-specific knowledge. It consists of two primary phases:

- **Retrieval Phase:** When a user submits a query, it is first processed by an embedding model (such as a sentence transformer). This model converts the query into a numerical format (a vector). The system then performs a semantic search across a *Vector Library* or database containing pre-processed, vectorized external data (e.g., enterprise documents, private emails, or updated web content). The most relevant documents or text chunks are retrieved based on vector similarity.
- **Generation Phase:** The originally submitted query is appended with the relevant information retrieved in the first phase. This combined prompt (query + context) is then fed into the base LLM. The model synthesizes this context to generate a highly accurate, tailored, and context-aware response.

1.2.5.2 Application and Benefits

One of the most significant advantages of RAG is its ability to interact with local, private datasets without the need to retrain or continuously fine-tune the massive base model. This makes the system more cost-effective, easily updatable, and significantly reduces the risk of AI "hallucinations" (instances where the model confidently generates false information).

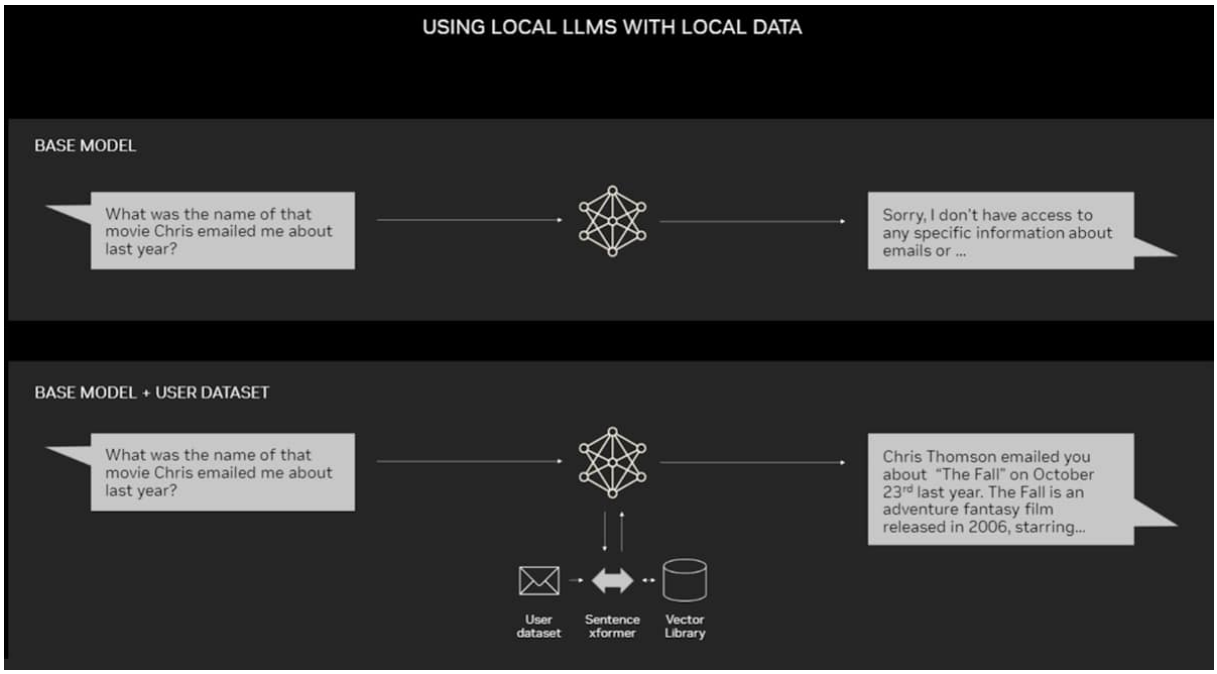


Figure 1.2: An example application for RAG on a PC, demonstrating the difference between a standalone base model and a RAG-enhanced model[1].

Figure 1.2 illustrates a practical application of RAG using local data. In the top scenario, a standard base model is asked a highly specific, personal question: *"What was the name of that movie Chris emailed me about last year?"* Lacking access to the user's private data, the model fails to provide a meaningful answer. In the bottom scenario, the system is augmented with a user dataset. The query is processed through a sentence transformer to interact with a vector library containing the user's emails. The retrieved context allows the base model to successfully identify the movie "The Fall" from the specific email, demonstrating RAG's power in personalizing and grounding AI interactions.

1.2.6 Multi-Agent Systems (MAS)

A Multi-Agent System (MAS) [15] is a distributed paradigm in artificial intelligence comprising multiple autonomous, interacting computational entities, known as agents, situated within a shared environment. Unlike traditional single-agent systems that rely on centralized control and operate in isolation, a MAS leverages decentralized decision-making. Within this framework, agents collaborate, coordinate, and sometimes compete to achieve individual or collective objectives. This collective behavior allows a MAS to tackle large-scale, complex, and dynamic tasks that would otherwise overwhelm a monolithic program.

1.2.6.1 Core Components and Workflow

The architecture of a multi-agent system fundamentally relies on three core elements[16]:

- **Agents:** These are the active, autonomous decision-making entities within the system. Each agent perceives its local surroundings and operates independently based on its assigned role and available information. In modern AI applications, an agent's reasoning is frequently powered by a Large Language Model (LLM), acting as its "brain" to understand complex intents, perform multi-step reasoning, and execute logic-based actions.
- **Environment:** Represents the computational or physical infrastructure where agents coexist. It defines the operational boundaries, manages resource availability, and enforces the rules of interaction. Beyond direct communication, the environment enables indirect coordination mechanisms where agents interact by modifying their shared state.
- **Interaction Mechanisms:** To function as a cohesive unit, agents rely on communication protocols and Agent Communication Languages (ACLs), Modern MAS also heavily rely on *orchestration*, a managed, goal-oriented process where complex tasks are broken down into structured workflows, ensuring agents are invoked in the correct sequence and information flows seamlessly between them.

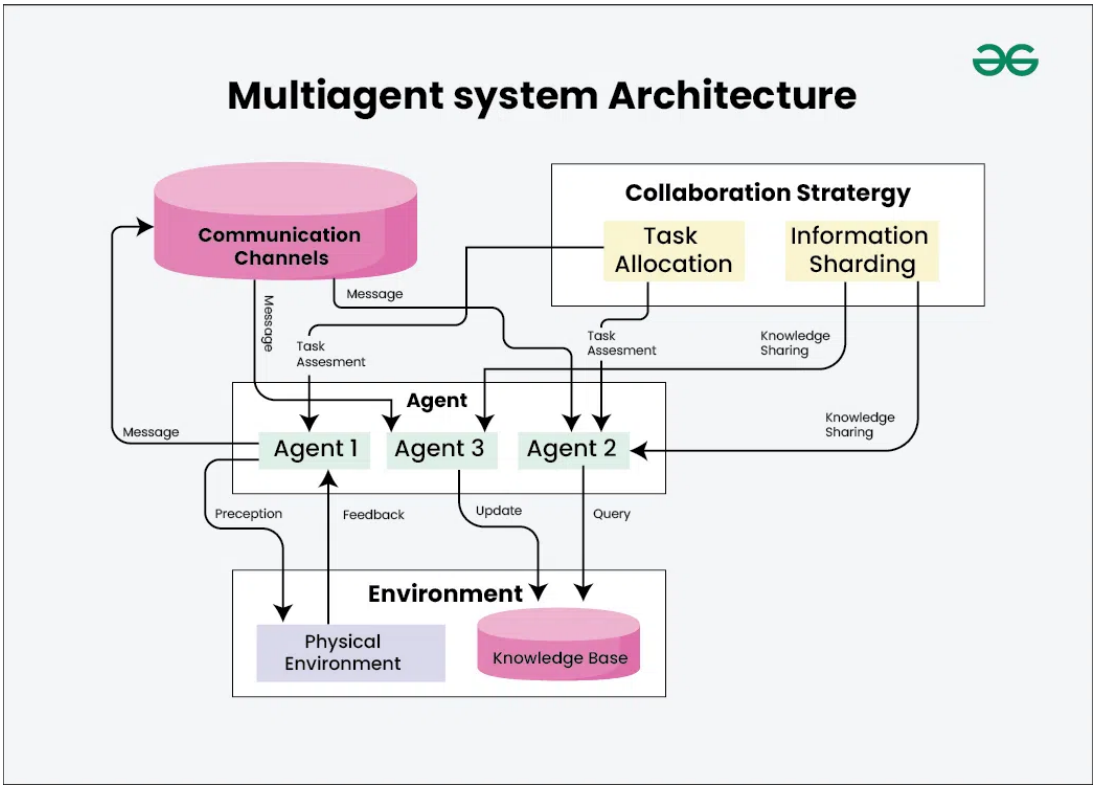


Figure 1.3: Architecture Overview of the Multi-Agent System[2].

1.2.6.2 Benefits and Challenges

Transitioning from a single-agent to a multi-agent architecture yields several significant advantages. Foremost among these are enhanced problem-solving capabilities, as tasks are distributed across highly specialized agents. Furthermore, MAS offer superior *scalability* (new agents can be integrated without system degradation) and *robustness* (if one agent fails, others can adapt and maintain system operations).

Despite these benefits, developing a MAS introduces considerable complexities. Challenges include managing communication overload among agents, mitigating unexpected emergent behaviors, and ensuring stringent security protocols. Additionally, in LLM-powered MAS, developers must navigate the high computational costs of API calls and the technical hurdle of factual grounding to prevent agents from hallucinating incorrect information.

1.3 Related Work

Recent advances in Large Language Models (LLMs) have significantly accelerated the automation of software testing, particularly in generating test artifacts from natural language specifications. This section reviews current research on leveraging LLMs to improve the effectiveness, efficiency, and fault-detection capabilities of automated tests. By analyzing these approaches, we provide insights into how AI technologies are transforming traditional testing paradigms and contributing to the development of more robust software systems.

1.3.1 Comprehensive Evaluation and Insights Into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation

Karpurapu et al. [17](2024) evaluate various LLMs (GPT-3.5, GPT-4, Llama-2, PaLM-2) for generating BDD acceptance tests from User Stories. Their approach relies exclusively on direct prompting (zero-shot and few-shot), without RAG or execution feedback loops, and validates the generated `.feature` files purely on syntax using Gherkin-lint. Results show that GPT-3.5 and GPT-4 produce nearly error-free Gherkin scenarios when utilizing few-shot prompting, whereas other models struggle significantly with syntax errors.

This research highlights the critical requirements for reliable automated test generation. Primarily, the findings indicate that utilizing advanced models in conjunction with few-shot prompting is essential to achieve strict syntactic compliance. Furthermore, the study underscores the necessity of integrating validation mechanisms as a primary safeguard against structural deviations. Crucially, it exposes a fundamental limitation in purely text-based generation: syntactic correctness does not inherently guarantee func-

tional validity against the application logic. This discrepancy suggests that static, text-only generation models are insufficient for ensuring reliable specifications, as they lack the capability to verify if the output aligns with the actual requirements and system behavior.

1.3.2 Increasing Test Coverage by Automating BDD Tests in Proofs of Concepts (POCs) using LLMs

Santos et al. [18] (2024) address technical debt in rapid prototyping by introducing AutoDevSuite, an LLM-based tool utilizing a LangChain one-shot prompting pipeline. It generates Python test scripts from a single prompt combining Gherkin User Stories and technical specifications, successfully increasing code coverage from 30% to nearly 100% in a real-world project. While concluding that Gherkin drastically simplifies prompt engineering, the authors highlight a critical flaw: the absolute necessity of expert manual code review to catch LLM-generated faulty or deprecated code.

This research confirms Gherkin as the optimal pivot format bridging business intent and code generation. However, the reliance on manual expert review highlights the limitations of linear pipelines, thereby justifying the need for a closed-loop framework to enhance test quality.

1.3.3 Unit Test Generation Multi-Agent AI System for Enhancing Software Documentation and Code Coverage

This research by Stojanović et al. [19] evaluates a multi-agent system based on the Microsoft AutoGen [20] framework for automated Python unit test generation. The architecture employs a hierarchical collaboration between three specialized agents: a *User Story Writer* (extracting Gherkin scenarios from code), a *Unit Test Writer*, and a *Code Executor*. The key contribution is the implementation of a feedback loop, where the executor validates tests in an isolated environment and transmits error traces back to the generator for iterative refinement.

A significant limitation of this approach is the absence of a Retrieval-Augmented Generation (RAG) mechanism, which forces the system to rely solely on the underlying model's internal training rather than external, real-time documentation. Without RAG to provide verified library specifications or coding patterns, the agents are susceptible to "contextual confusion," leading to syntax errors, infinite loops, or the misuse of built-in functions. This lack of external grounding explains why the current pass rate is capped at 85%, as the model operates via token prediction rather than deep logical verification from a retrieved knowledge base. Furthermore, the scope remains restricted to unit testing, which isolates individual components but fails to validate the complex interactions and integration flows between multiple services, potentially overlooking critical system-level vulnerabilities.

1.3.4 Discussion

Table 1.1 summarizes the key characteristics of the reviewed research works.

Table 1.1: Summary of research approaches for test generation

Study	LLMs Used	Test Types	Input Data	Approach Type	Prompt Type	Supported Testing Tasks
Karpurapu et al. (2024)	GPT-3.5-Turbo, GPT-4, Llama-2-13B, PaLM-2	Acceptance (BDD)	User Stories (CSV)	LLM-based	Zero-shot, Few-shot	Prompt formulation, Test scenario generation, Automated syntax validation
Santos et al. (2024)	Google Gemini, ChatGPT, Ollama	Acceptance and System testing	User Stories, Acceptance Criteria (Gherkin), technical specifications	LLM-based	One-shot	Test code generation, Coverage validation
Stojanović et al. (2024)	GPT-4o-mini	Unit testing	Python function or method code	Multi-agent-based	Not specified	Test generation, Test execution, Feedback reporting

While purely prompt-based methods effectively translate business needs into Gherkin, they lack execution validation, necessitating manual review to filter LLM hallucinations. Conversely, existing agentic execution loops ensure reliability but focus solely on unit tests, ignoring high-level business requirements.

1.4 Problem Statement

Based on the identified limitations, a fundamental gap emerges between business-level test specification and technically validated test execution, as current approaches either generate high-level test scenarios without ensuring their functional correctness or validate low-level unit tests without incorporating business requirements.

This fragmentation leads to a critical issue: **the inability to automatically produce test scenarios that are both semantically aligned with business needs and dynamically validated against the actual system behavior**.

In modern microservice-based architectures, this limitation is further amplified by system complexity, inter-service dependencies, and the need for rapid delivery cycles. Consequently, manual test design becomes a bottleneck, forcing teams to prioritize speed over coverage and increasing the risk of undetected defects in production.The core problem can therefore be reformulated as follows:

How can we automatically generate test scenarios that are both business-driven and technically validated, while ensuring comprehensive coverage (including edge cases) without slowing down the development lifecycle?

To address these limitations, a new approach is required—one that combines the strengths of LLM-based generation with execution-driven validation in an iterative framework.

1.5 Proposed Solution: VORTEX - A Multi-Agent BDD End To End System Approach to Software Test Automation

To overcome the latency of manual Behavior-Driven Development (BDD) test creation and address the critical trade-off between deployment speed and test coverage, we propose **VORTEX (Validation Engine for Robust Testing & Execution of Microservices)**, an automated testing framework powered by collaborative autonomous agents.

1.5.1 Core Concept and Philosophy

At the heart of our solution is a Multi-Agent System (MAS) designed to mimic the dynamics of a real-world software quality assurance team. To handle the entire testing lifecycle, we leverage the MAS philosophy of *separation of concerns*. In this ecosystem, the proposed solution is designed to support the end-to-end and the BDD automation of the testing process. It enables the transformation of business requirements into structured test scenarios, ensures their syntactic correctness, generates executable test code based on available system specifications, and validates these tests through automated execution. By structuring the process into well-defined functional steps, the system improves consistency, reduces manual effort, and enhances the overall reliability of the testing workflow.

1.6 Functional and Non-Functional Requirements

To ensure the proposed multi-agent-based system effectively addresses the challenges of modern software testing, its design and implementation are guided by a specific set of functional and non-functional requirements.

1.6.1 Functional Requirements

Functional requirements define the core capabilities and behaviors the system must exhibit to successfully automate the Behavior-Driven Development (BDD) testing lifecycle:

- **Requirement Ingestion and Analysis:** The system must be able to ingest raw business requirements (User Stories) and autonomously analyze them to design comprehensive test cases, identifying both nominal workflows and edge cases.

- **Scenario Generation and Validation:** The system must translate conceptual test cases into standard Gherkin syntax (Given-When-Then format) and validate this syntax against established BDD conventions before proceeding.
- **Automated Test Code Generation:** The system must generate executable test code (e.g., JUnit, Cucumber) by mapping the validated Gherkin steps to the corresponding system interfaces or API documentation (e.g., Swagger/OpenAPI).
- **Test Execution and Reporting:** The framework must automatically orchestrate the execution of the generated test scripts within a designated environment and generate detailed execution reports indicating pass/fail statuses and coverage metrics.
- **Autonomous Self-Healing:** Upon encountering a test failure caused by a script anomaly or logic error, the system must autonomously analyze the execution logs, correct the underlying test code, and re-execute the test without human intervention.
- **Contextual Knowledge Retrieval (RAG):** The system must implement a Retrieval-Augmented Generation (RAG) mechanism to query external technical resources, such as API documentation, architectural schemas, and internal coding standards. This ensures that the generated test scenarios and scripts are grounded in real-time technical truth, minimizing hallucinations and improving the accuracy of the testing logic for complex enterprise services.

1.6.2 Non-Functional Requirements and Their Realization in the Proposed Approach

Non-functional requirements specify the quality attributes, performance goals, and architectural constraints of the system:

- **Performance and latency:** To respect the strict time-to-market constraints of modern CI/CD pipelines, the multi-agent orchestration must generate and execute tests rapidly, ensuring that the automated quality assurance phase does not become a bottleneck.
- **Reliability and accuracy:** Because the agents rely on Large Language Models (LLMs), the system must incorporate grounding techniques (such as RAG) to minimize hallucinations and ensure that generated code and scenarios strictly adhere to the provided documentation and context.
- **Modularity and extensibility:** The multi-agent architecture must be highly modular, allowing developers to easily add new specialized agents, update existing prompts, or swap the underlying LLM without disrupting the entire workflow.

1.7 Conclusion

This chapter has established the foundational context and theoretical groundwork for the project. We began by exploring the core concepts of Behavior-Driven Development (BDD), Multi-Agent Systems (MAS), and Retrieval-Augmented Generation (RAG), which collectively serve as the technological pillars of our approach. By reviewing the state-of-the-art in software testing, we identified a critical industry gap: the significant latency introduced by manual test creation, which fundamentally clashes with the speed of modern distributed architectures and CI/CD pipelines.

To address this bottleneck, we formulated a clear problem statement and introduced an innovative conceptual solution: an autonomous, multi-agent framework designed to automate the entire functional testing lifecycle. Finally, we defined functional and non-functional requirements that will guide the development and evaluation of this framework.

With the project's vision, boundaries, and theoretical foundations clearly established, the following chapter will delve into the technical architecture. It will detail the specific frameworks, data flows, and structural design required to bring this collaborative multi-agent concept to life.

Chapter 2

System Architecture and Design

2.1 Introduction

This chapter presents our main contribution, which aims to overcome several limitations identified in the existing literature on automated testing for complex systems. Our work proposes a structured and intelligent approach based on a multi-agent paradigm to improve the generation and validation of test artifacts.

To provide a clear and organized presentation, the chapter is divided into two main parts.

First, we introduce the *System Under Test* (SUT), which serves as the application context used to illustrate and evaluate our approach. This part focuses on its functional analysis and requirements, as well as its overall structure through a domain model, providing a clear understanding of the system from a conceptual perspective.

Second, we present our proposed contribution: the *Test System*. This part describes the fundamental principles of our approach, including the overall pipeline and the collaborative behavior of the multi-agent system. The emphasis is placed on the general workflow and the underlying concepts that enable automated test generation and validation.

This organization ensures a clear distinction between the application context and the proposed solution, while highlighting the essential concepts that form the foundation of our work.

2.2 The System Under Test Overview

Before detailing the automated testing framework, it is crucial to understand the target application being tested. The System Under Test (SUT) serves as the baseline for evaluating the effectiveness of our multi-agent approach. For this project, the SUT is a Leave Request Management application. To ensure modularity and scalability, the SUT is architected as a distributed system comprising two primary microservices: **User Management** and **Leave Management**.

2.2.1 Functional Analysis and Requirements

The functional analysis phase defines the precise needs and behaviors expected from the SUT. To conceptualize this, the system's requirements are synthesized visually through a **Use Case Diagram**. As illustrated in Figure 2.1, the system interacts with three primary actors: the Employee, the Team Lead and the Administrator. The diagram maps these actors to their respective privileges across the two microservices, highlighting core functionalities such as authentication, managing personal leave requests, and overseeing subordinates' requests.

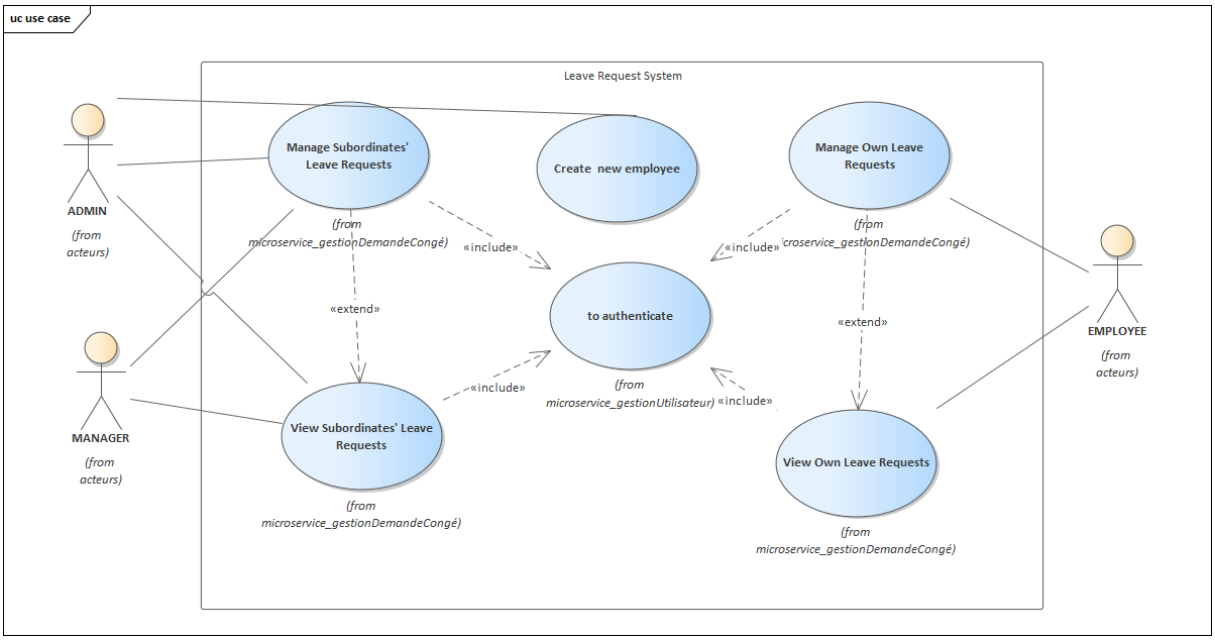


Figure 2.1: Use Case Diagram of the Leave Request Management System, distributed across User and Leave Management microservices.

While the Use Case diagram provides a high-level abstraction of the system's boundaries, automated Behavior-Driven Development (BDD) testing requires granular, textual requirements. Therefore, these overarching use cases are decomposed into a Product Backlog consisting of precise user stories, detailed business rules, and acceptance criteria.

2.2.1.1 Product backlog extract

The following list outlines the core user stories categorized by their respective microservice:

Microservice: Leave Management

- **US-01: Add Leave Request:** As an employee, I want to create a new leave request, so that I can submit it for validation based on the selected leave type.
- **US-02: Cancel Leave Request:** As an employee, I want to cancel a pending leave request, so that I can withdraw a request that has not yet been granted.

- **US-03: Approve Leave Request:** As an Administrator or Team Lead, I want to approve an employee's leave request, so that I can officially validate their absence according to the defined approval chain.
- **US-04: Refuse Leave Request:** As an Administrator or Team Lead, I want to refuse an employee's leave request, so that I can properly manage absences when conditions or justifications are not satisfactory.
- **US-05: View Leave Requests (Access Control):** As a system user (Employee, Team Lead, or Administrator), I want to view and filter leave requests based on my access rights, so that I can track my own absence history or validate my team's requests.

Microservice: User Management

- **US-06: Log In:** As a user (Administrator, Team Lead, or Employee), I want to log in to the application, so that I can access the interface and features corresponding to my role.
- **US-07: Change Password After Reset:** As a User, I want to define a new personal password after using a temporary one, so that I can secure my account with a credential I know.
- **US-08: Logout:** As an authenticated user, I want to log out from the system, so that my session is securely terminated.
- **US-09: Create Employee Account:** As an Admin, I want to create a new employee account, so that I can integrate them into the company's management system.

2.2.1.2 Business Rules and Acceptance Criteria

To enable the multi-agent system to generate exhaustive Gherkin scenarios (covering both happy paths and edge cases), each User Story must be accompanied by strict **Business Rules** and **Acceptance Criteria**. This granular level of detail is exactly what the autonomous agents require to deduce logical test permutations and generate robust automation scripts.

To illustrate this depth of specification, below is the complete breakdown for the "Approve Leave Request" functionality, presented in a structured format:

Target User Story (US-03): Approve Leave Request

As an Administrator or Team Lead, I want to approve an employee's leave request, so that I can officially validate their absence according to the defined approval chain.

Business Rules:

- The user must belong to the request's approval chain.
- A request can only be approved if its status is "Pending" or "In Progress".
- An approver cannot validate a request they have already previously approved.
- Validation follows a hierarchical chain: as long as the administrator has not approved, the request remains "In Progress". When the final approver grants approval, the request status becomes "Granted".
- Upon final validation, the leave balance is deducted according to the type (annual, authorization, recovery).

Acceptance Criteria:

- The user can view the complete details of the request before validation.
- The user can add an observation (optional).
- If the user is not authorized to validate this step, an error message is displayed: *"You are not authorized to modify the status of this request."*
- If the request is not in a valid state ("Pending" or "In Progress"), validation is blocked.
- If the user has already validated previously, the validation is refused.
- If the user is the final approver: the request status changes to "Granted" and the leave balance is adjusted according to the rules of the leave type.
- Otherwise: the status changes to "In Progress" and the system marks the manager's validation as TRUE.
- After validation, the system displays *"Request granted successfully."*

2.2.2 Static Design and Domain Model

To support the functional requirements and user stories outlined previously, the internal structure of the System Under Test is modeled through a class diagram. This domain model provides a static view of the core business entities and their structural relationships.

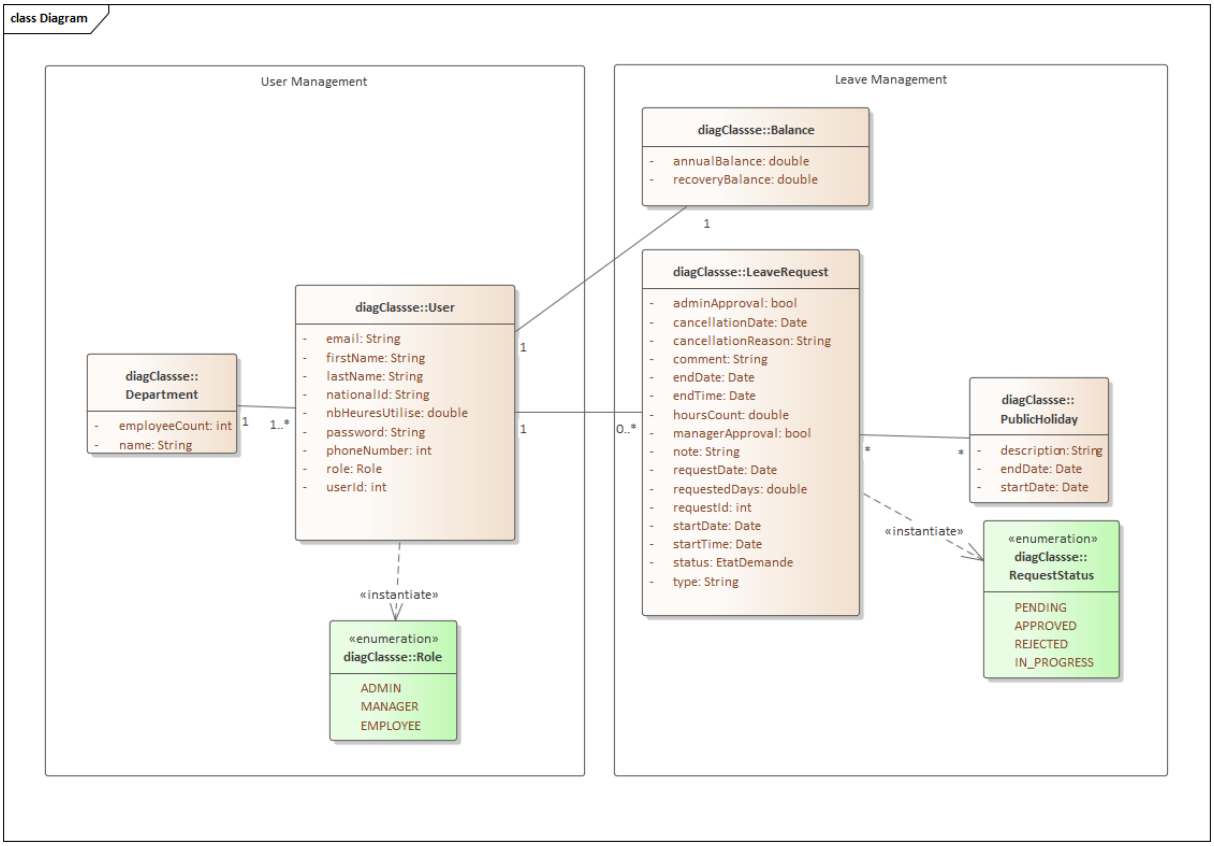


Figure 2.2: Class Diagram illustrating the domain model of the Leave Request Management System.

As shown in Figure 2.2, the system's entities are logically grouped to reflect the microservice-based architecture. The left section focuses on the User Management domain, which is structured around the **User** entity, its associated roles, and department affiliations. The right section illustrates the Leave Management domain, encompassing entities such as **Balance**, public holidays, and the central **LeaveRequest** class.

Among these, the **User** and **LeaveRequest** classes are the most critical components of the system. They encapsulate the essential business logic, constraints, and state transitions (e.g., moving a request from "PENDING" to "IN_PROGRESS" or adjusting leave balances). Consequently, the methods governing these two core entities are the primary targets for the validation rules and automated test scenarios generated by our multi-agent test system.

2.3 Multi-agent-based Test System

This section focuses on the internal design of the multi-agent Test System, detailing the role of each agent and the principles behind their interaction and coordination.

Figure 2.3 illustrates the proposed multi-agent pipeline for automated test generation. The system follows a structured workflow that transforms high-level inputs into validated test artifacts through a sequence of coordinated processing steps.

The process starts from two main inputs, **Business Logic** and **Technical & Environment Context**, which are used to generate and refine test scenarios. These scenarios pass through a series of validation and transformation stages, ensuring their correctness before execution.

A set of decision points guides the pipeline execution, allowing iteration in case of validation issues or test failures. The system also includes a **Knowledge Layer** that enriches the process by providing contextual information throughout the workflow.

Finally, the pipeline produces several outputs, including validated test scenarios, executable test cases, and execution results with coverage information.

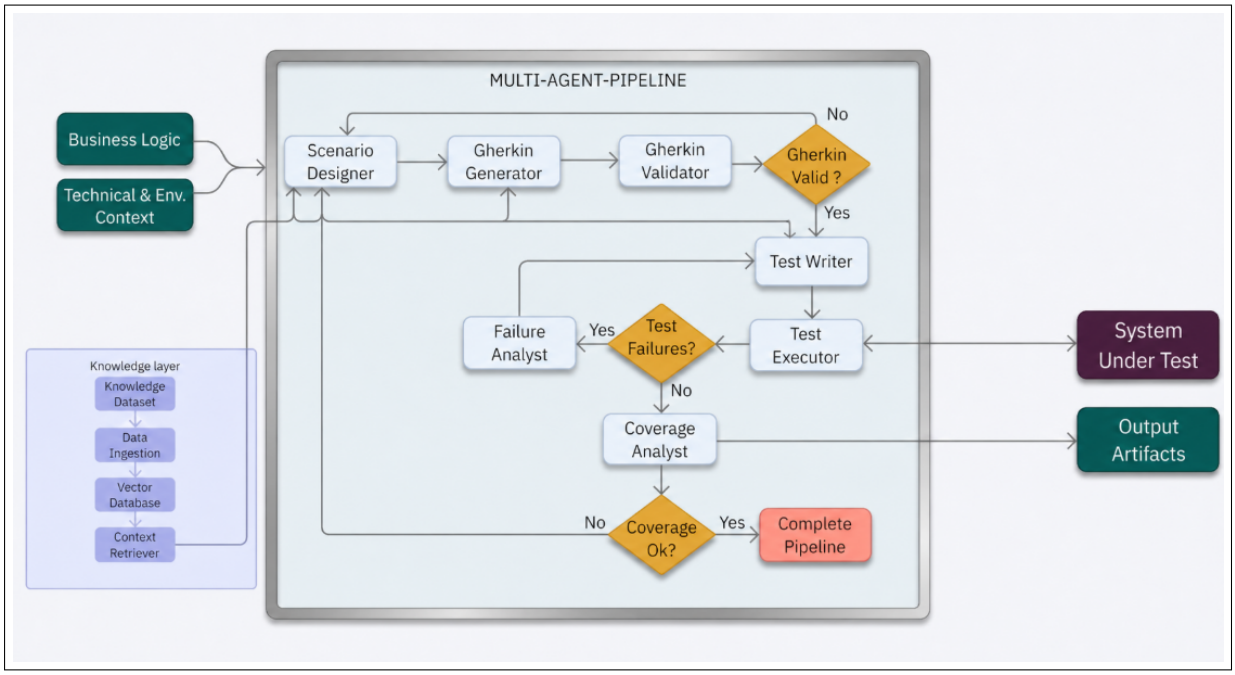


Figure 2.3: Overview of the multi-agent-based test system for automated BDD test generation.

2.3.1 Scenario Designer Agent

The **Scenario Designer Agent** serves as the strategic starting point of the pipeline, operating just before the Gherkin generation phase. Its primary role is to bridge the gap between technical API contracts and actual business logic. It achieves this by ingesting

two main categories of inputs: *business logic and technical & environment context*. The business logic includes elements such as user stories, business requirements, and business rules, while the technical and environment context covers API specifications, the service matrix, which is a centralized configuration defining system services, their execution parameters, and their interdependencies, and execution environment constraints.

Because generating tests solely from technical specifications often results in weak test coverage, this agent combines both perspectives to produce highly structured, context-aware test scenarios in a Given/When/Then format. These generated scenarios comprehensively cover happy paths, error states, edge cases, security constraints, and cross-service integrations. By producing a rich, JSON-formatted scenario blueprint, it feeds directly into the *Gherkin Generator Agent*, ensuring robust test coverage and identifying critical business flows that might otherwise be overlooked.

To support this improvement cycle, the agent leverages a Retrieval-Augmented Generation (RAG) mechanism prior to producing its output. This mechanism is important as it allows the agent to retrieve relevant contextual knowledge and reuse past information, ensuring that generated scenarios are not only based on the current inputs but also enriched with previously validated domain knowledge. This leads to more accurate, consistent, and context-aware test scenarios, particularly in complex cases where business rules and technical constraints must be aligned.
In the next chapter, we provide further details on the internal architecture and working principles of the Scenario Designer Agent.

Scenario Designer Agent Prompt: The Scenario Designer Agent relies on a structured CO-STAR prompt that instructs the model to produce branch-coverage-aware test scenarios from business requirements and API specifications. The prompt enforces strict JSON output, endpoint fidelity against the Swagger catalog, and a mandatory type distribution across happy path, error, edge, and security scenarios to ensure downstream compatibility with the Gherkin Generator.

Prompt Template – Scenario Designer Agent

Context
You are a senior QA scenario designer for API and end-to-end testing, specialized in branch-coverage-aware test design targeting validation branches, state transitions, and role-based access controls.

Objective
Generate a JSON-formatted set of Given/When/Then test scenarios for the `{service_name}` microservice, covering all critical business flows and exposing both sides of every conditional branch.

Style
Precise and technical. Field values must be concrete – avoid vague expressions like

"valid data" without specifying what makes them valid.

Tone

Formal and unambiguous. Each scenario field must be a single, self-contained, actionable sentence.

Audience

Downstream automation agents and QA engineers translating scenarios into executable Gherkin feature files. **Inputs**

- User Story: `{user_story}`
- Service Requirements: `{service_requirements}`
- Swagger Endpoint Catalog: `{swagger_summary}`
- Coverage Feedback (optional): `{coverage_feedback}`

Rules

- Return JSON only – no markdown fences or prose.
- Use only endpoints from the Swagger catalog; never invent paths or actors.
- The `when` field must include the exact HTTP method and endpoint path.
- Generate 8–16 scenarios normally, or 4–10 if coverage feedback is provided.
- Type distribution: **happy_path** ≥30%, **error_case** ≥30%, **edge_case** ≥20%, **security** ≥20%.

Response Format

```
{ "scenarios": [{
  "title": "...", "endpoint": "/api/...", "method": "POST",
  "given": "...", "when": "POST /api/... invoked with ...",
  "then": "...", "test_type": "happy_path|error_case|edge_case",
  "priority": "P0|P1|P2", "service": "{service_name}",
  "is_integration": false }] }
```

2.3.2 Gherkin Generator Agent

The **Gherkin Generator Agent** bridges the gap between raw functional requirements and structured test specifications. It takes as input the structured output produced by the previous Scenario Designer Agent, along with the associated API contract. Based on this information, the agent produces well-formed feature files written in Gherkin syntax, organizing test coverage around the *Given–When–Then* format. Each generated file captures a complete end-to-end user journey, spanning all involved microservices and covering both nominal and error scenarios. The resulting feature files serve as the primary input artifact for the downstream test generation stages of the pipeline.

The Gherkin Generator Agent leverages a Retrieval-Augmented Generation (RAG)

mechanism prior to producing its output. This mechanism is important as it allows the agent to use relevant contextual information from previous test artifacts and domain knowledge, ensuring that the generated Gherkin feature files are more consistent, complete, and aligned with both business requirements and API specifications.
In the next chapter, we provide further details on the internal architecture and working principles of the Gherkin Generator Agent.

Gherkin Generator Agent Prompt : The Gherkin Generator Agent relies on a structured CO-STAR prompt that provides the language model with the necessary context to generate well-formed Gherkin feature files. The prompt is organized into clearly defined sections, including the agent role, the runtime inputs (user story, API contract, and extracted error messages), and explicit output formatting rules. This structured design ensures that the generated feature files are consistent, syntactically valid, and suitable for use by downstream components of the pipeline.

Prompt Template – Gherkin Generator Agent

Context
You are a BDD testing expert generating Gherkin feature files for end-to-end validation.
Objective
Generate a complete and valid `.feature` file describing the full user journey.
Audience, Style & Tone
The output is intended for QA engineers. Use a formal, structured, and technical style with no natural language outside valid Gherkin syntax. **Inputs**
- User Story: `{user_story}`
- API Specification: `{api_spec}`
- Error Messages: `{error_messages}`

Instructions
- Follow strictly the Given–When–Then format.
- Use only elements defined in the API specification.
- Copy error messages exactly (no modification).
- Describe business behavior only (no technical details).

Required Coverage
- One happy path scenario
- One scenario per error message
- One unauthorized access scenario
- One Scenario Outline

Response Format

```
Feature: <feature name>
Scenario: <scenario name>
```

```
Given <precondition>
When  <action>
Then  <expected result>
```

2.3.3 Gherkin Validator Agent

The **Gherkin Validator Agent** ensures the quality and correctness of the feature files generated by the previous agent before they are used in the test generation stage. It validates the syntactic correctness of the Gherkin structure and ensures that scenarios follow a coherent Given–When–Then format.

It also checks the logical consistency of the scenarios and evaluates their coverage with respect to the original user story and its requirements. This ensures that all relevant acceptance criteria are properly addressed. The output of this agent is a set of validated feature files that are consistent, complete, and ready for the next stage of the pipeline.

If the Gherkin syntax is invalid, the Gherkin Validator Agent sends its report back to the Gherkin Generator agent, which regenerates the output by correcting the errors identified in the initial iteration.

Gherkin Validator Agent Prompt : The Gherkin Validator Agent relies on a structured CO-STAR prompt that instructs the large language model to evaluate the quality and completeness of the generated feature files. The prompt provides the model with the original user story requirements alongside the Gherkin content to be assessed, and directs it to produce a structured validation report covering coverage gaps, semantic inconsistencies, and improvement suggestions.

Prompt Template – Gherkin Validator Agent

Context

You are a software quality assurance expert specialized in BDD test coverage analysis. You evaluate Gherkin feature files against functional requirements to ensure completeness and correctness.

Objective

Analyze the provided Gherkin scenarios against the original requirements and produce a structured validation report assessing coverage, coherence, and completeness.

Audience, Style & Tone

The output is intended for QA engineers and test leads. Use a formal, analytical, and structured style with precise and actionable feedback.

Inputs

- Requirements: `{requirements_text}`
- Gherkin Scenarios: `{gherkin_content}`

Instructions

- For each requirement, verify whether a corresponding scenario exists.
- Identify scenarios that violate the Given–When–Then ordering.
- Report missing scenarios as concrete, actionable items.
- Calculate the coverage score based on covered versus total requirements.
- Do not mark a scenario as missing if it clearly exists in the Gherkin content.

Response Format

```
{
 "coverage_score": <integer 0-100>,
 "missing_scenarios": [<list of uncovered requirements>],
 "issues": [<list of detected problems>],
 "suggestions": [<list of improvement recommendations>]
}
```

2.3.4 Test Writer Agent

The **Test Writer Agent** bridges the gap between the validated test specifications and their concrete implementation. It takes the Gherkin feature files produced by the previous stages and translates each scenario into executable test artifacts, ensuring full coverage of the end-to-end user workflows described in the original requirements. The Test Writer Agent also produces the necessary project configuration to make the generated test suite self-contained and immediately executable. Its output serves as the foundation for the test execution phase that follows in the pipeline.

To support this improvement cycle, the agent leverages a Retrieval-Augmented Generation (RAG) mechanism prior to producing its output. This mechanism is important as it allows the agent to retrieve relevant contextual knowledge and reuse past information, ensuring that generated test artifacts are not only based on the current Gherkin scenarios but also enriched with previously validated implementation knowledge. This leads to more accurate, consistent, and context-aware test implementations, particularly in complex cases where step definitions, business rules, and technical constraints must be correctly aligned.

In the next chapter, we provide further details on the internal architecture and working principles of the Test Writer Agent.

Test Writer Agent Prompt : The Test Writer Agent relies on a structured CO-STAR prompt that guides the large language model in translating validated Gherkin scenarios into executable Java test code. The prompt supplies the model with the feature file content and the API contract, and directs it to produce complete and runnable step definition classes that concretely implement each scenario step through appropriate service calls.

Prompt Template – Test Writer Agent

Context

You are a senior Java test engineer specialized in BDD automation. You implement Cucumber step definitions that interact with REST APIs to validate microservice behavior.

Objective

Generate complete and executable Java step definition classes that implement every Given–When–Then step from the provided Gherkin feature file, using RestAssured to invoke the appropriate API endpoints.

Audience, Style & Tone

The output is intended for QA automation engineers. Use a formal and technical style producing only valid, compilable Java code with no natural language outside comments.

Inputs

- Gherkin Feature File: `{gherkin_content}`
- API Specification: `{api_spec}`
- Base Service URL: `{base_url}`

Instructions

- Generate one `@Given`, `@When`, or `@Then` method per unique Gherkin step.
- Use the API specification to determine the correct endpoint, HTTP method, and request body for each step.
- Never hardcode credentials – use environment variables.
- Ensure zero undefined steps – every step in the feature file must have a matching method.
- Produce only valid, compilable Java – no markdown, no explanations outside comments.

Response Format

```
@Given("<step pattern>")
public void <methodName>() {
 // RestAssured call
}
@When("<step pattern>")
public void <methodName>() {
```

```
 response = given()
 .baseUri(BASE_URL)
 .header("Authorization","Bearer "+jwtToken)
 .when().<method>("<endpoint>")
 .then().extract().response();
}
@Then("<step pattern>")
public void <methodName>() {
 // assertion
}
```

2.3.5 Test Executor Agent

The **Test Executor Agent** serves as the operational engine of our automation pipeline. Its primary responsibility is to take the executable test scripts generated by the *Test Writer Agent* and autonomously run them against the System Under Test (SUT). It handles environment preparation, simulates requests to the distributed microservices, and executes the BDD end-to-end scenarios. Throughout this process, it monitors system behavior, compiles successes and failures, and generates a comprehensive execution report.

If a test fails due to an issue in the test code itself rather than the system under test, the Test Executor Agent forwards its report to the Failure Analyst Agent for correction and regeneration of the affected tests. Otherwise, the execution report is sent to the Coverage Analysis Agent to perform the necessary evaluations, metrics computation, and statistical analysis.

2.3.6 Failure Analyst Agent

The **Failure Analyst Agent** acts as the diagnostic brain of the automation pipeline. Once a test execution completes, this agent reviews the results and logs to understand exactly why the tests failed.

Instead of simply reporting that an error occurred, the Failure Analyst Agent categorizes the root cause—distinguishing between transient issues, actionable test errors, and critical environment blocks. Based on this diagnosis, the Failure Analyst Agent makes a smart decision: it either triggers an automated "self-healing" retry to resolve the issue or flags it for human intervention if the problem cannot be fixed automatically.

In essence, it turns raw failure data into clear, actionable intelligence, ensuring the

pipeline only wastes resources on retries that have a high probability of success.

The output of the Failure Analyst Agent is then forwarded to the Test Writer Agent for test correction, refinement, and regeneration when necessary.

2.3.7 Coverage Analyst Agent

The **Coverage Analyst Agent** acts as an automated post-test quality gate, synthesizing complex coverage metrics into actionable insights. It takes the output produced by the Test Executor Agent as its primary input, intelligently parsing data from various reports and managing the lifecycle to ensure consistency.

By validating findings against predefined quality thresholds, it identifies gaps and delivers structured, ready-to-use reports in standardized formats for seamless dashboard and pipeline integration.

The output of the Coverage Analyst Agent is an aggregated artifact that includes *behavior files*, the generated *test codebase*, *test execution reports*, overall *coverage metrics*, as well as a *detailed coverage analysis report*, providing a complete view of test quality and system validation results.

If the coverage and metrics results are not satisfactory, the Coverage Analyst's report is sent back to the Scenario Designer Agent to rework and regenerate the scenarios, taking into account the identified issues that need to be addressed. Otherwise, the Coverage Analyst's output is directly forwarded and displayed on the user interface.

2.3.8 Retrieval-Augmented Generation (RAG)

To further enhance the quality and relevance of the generated artifacts, the system integrates a Retrieval-Augmented Generation component. Rather than relying solely on the information provided at runtime through the user story and the API contract, the RAG mechanism enriches the context available to the agents by incorporating knowledge retrieved from three external corpora: (1) a collection of real-world user stories from the Zenodo dataset `z13880060`, (2) GUI-level Java JUnit test cases extracted from the `E2EGit` database, and (3) the `GivenWhenThen` dataset, which contains feature files, step definitions, and linked system code files collected from open-source repositories. Together, these sources provide the system with concrete BDD examples, proven test patterns, and domain-relevant scenario structures, allowing the agents to produce outputs that are more coherent, more precise, and better aligned with established testing practices.

At a conceptual level, the RAG pipeline operates through a sequence of successive stages. The process begins with **chunking**, which consists of breaking down the available

reference documents into smaller, semantically meaningful units that can be individually indexed and compared. These chunks then undergo **chunk embedding**, where each unit is transformed into a numerical representation that captures its semantic content. In parallel, when an agent formulates a request, **query embedding** applies the same transformation to the input query, producing a comparable representation. A **similarity search** is then performed to measure the semantic closeness between the query and the indexed chunks, identifying and retrieving the most relevant candidates. The selected chunks may then undergo an optional re-ranking phase, where a more precise relevance score is applied to retain only the most relevant pieces of information. The refined selection is then assembled during the **context construction** stage, producing a coherent and focused knowledge block. Finally, this constructed context is **injected into the agent's prompt**, where it complements the structural information from the API specifications and guides the agent toward more accurate and domain-aware outputs.

Within the pipeline, the RAG component acts as a supporting layer for the agents, supplying them with additional contextual knowledge that goes beyond the runtime input alone. This enrichment contributes to improving the precision of the generated scenarios, strengthening their consistency with the domain requirements, and ensuring that the produced test artifacts reflect a deeper understanding of the system under test.

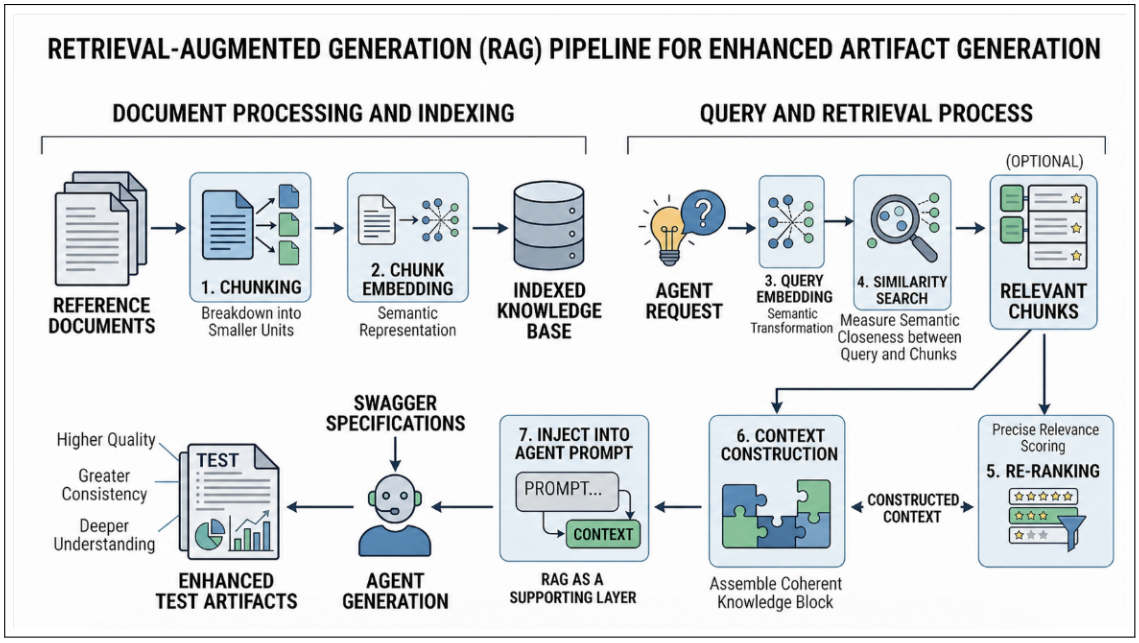


Figure 2.4: Retrieval-Augmented Generation (RAG) Pipeline for Enhanced Artifact Generation.

RAG Retrieval Prompt : The RAG component relies on a structured prompt that instructs the language model on how to exploit the retrieved context. The prompt ensures that the retrieved chunks are used strictly as supporting knowledge and never override the primary sources of truth defined by the Swagger contract and the user story.

Prompt Template – RAG Context Injection

Context

You are a domain-aware assistant supporting a BDD test generation pipeline. You have access to relevant knowledge retrieved from an indexed document base to enrich your understanding of the system under test.

Objective

Use the retrieved context to improve the relevance and domain accuracy of the generated output, without contradicting or replacing the information provided by the API contract and the user story.

Audience, Style & Tone

The output is intended for QA engineers. Maintain a formal and technical style consistent with the rest of the generated artifacts.

Inputs

- Retrieved Context (RAG): `{rag_context}`
- User Story: `{user_story}`
- API Specification: `{api_spec}`

Instructions

- Use the retrieved context as supplementary domain knowledge only.
- Do not treat retrieved chunks as API contract – Swagger remains the source of truth.
- If the retrieved context contradicts the API specification, ignore it.
- Integrate relevant domain insights naturally into the generated output.

Response Format

Follow the output format defined by the currently executing agent prompt. The RAG context is an enrichment layer and does not alter the expected structure of the response.

2.4 Conclusion

This chapter presented the design and architecture of the proposed multi-agent test automation system. We first introduced the System Under Test, which provides the functional requirements and domain model used as input for the pipeline.

We then described the overall multi-agent structure, where each agent contributes to transforming high-level business requirements into validated and executable test artifacts. The system is enhanced by three feedback loops: a validation loop ensuring syntactic and semantic correctness, a self-healing mechanism for handling failures, and a coverage improvement cycle to enhance test completeness. In addition, a shared Retrieval-Augmented Generation (RAG) layer enriches the pipeline with relevant domain knowledge.

Chapter 3

Implementation and Experimental Results

3.1 Introduction

This chapter presents the implementation of the proposed multi-agent testing framework and evaluates its performance through a series of controlled experiments. It is organized into four main sections.

First, we describe the prototype architecture and its implementation, covering the global system pipeline, the technical stack, and the integration of the Retrieval-Augmented Generation (RAG) component. Next, we present the Graphical User Interface (GUI) developed to interact with the system. We then define the evaluation metrics and experimental setup used to assess the framework, before conducting a thorough performance analysis across three successive versions of the system. Finally, we provide a comparative analysis between expert-written and LLM-generated Gherkin scenarios, highlighting the strengths and limitations of our approach.

3.2 Prototype and Architecture

This section presents the concrete implementation of the proposed approach, detailing the system architecture, the communication flows between components, and the key design decisions made during development.

3.2.1 System Pipeline: Architecture Overview

Figure 3.1 illustrates the system architecture and the three distinct communication flows: direct HTTP interaction for the Frontend, technical monitoring for the Test System, and the delegated business testing process triggered via CLI.

1. Frontend (React Application) User interface layer built with Vite + React + CSS. **Communication:** Direct HTTP REST requests to backend business endpoints for

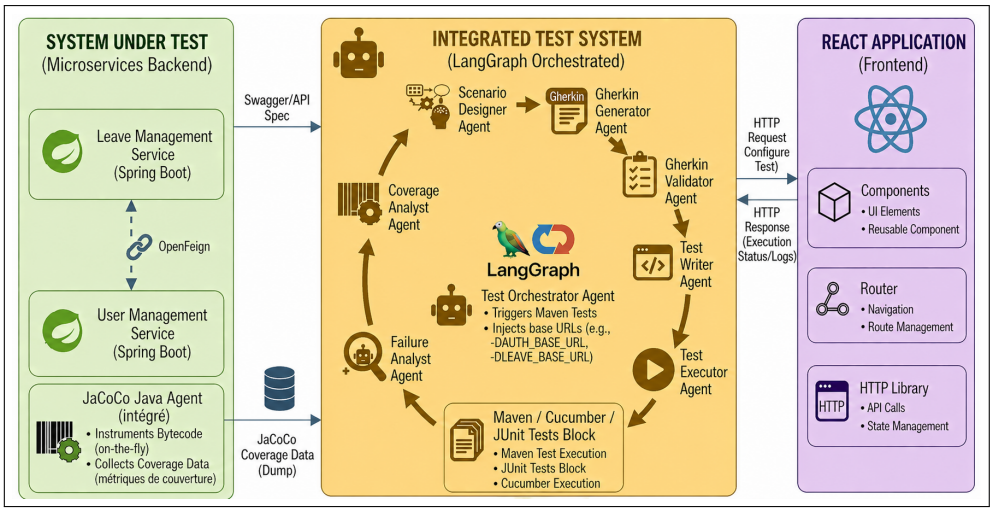


Figure 3.1: Architecture and communication flows

health checks, Swagger specification retrieval, and state polling during test execution.

2. System Under Test (Backend - Spring Boot Microservices) Core application consisting of multiple Spring Boot microservices exposing business services and monitoring tools. Services are instrumented with the JaCoCo agent in `tcpserver` mode, dumping real backend coverage data to the Test System. **Communication:** Receives HTTP business requests exclusively from the generated Java test suite. Python queries health, Swagger, and JaCoCo endpoints only.

3. Integrated Test System (Python QA Orchestrator - LangGraph) A LangGraph workflow coordinating a pipeline of specialized agents covering the full testing lifecycle: from API specification analysis and test scenario design, through test code generation and execution, to coverage collection and reporting.

3.2.2 Prototype Implementation and Tooling

This section presents the technologies selected for both the System Under Test and the multi-agent Test System. Each layer was chosen to ensure modularity, scalability, and compatibility with the overall pipeline architecture.

3.2.2.1 Tools Used for the System Under Test

The System Under Test is designed using a microservice-based architecture to ensure high cohesion, loose coupling, and scalability. To implement this distributed architecture efficiently, we selected a modern, enterprise-grade technology stack:

Java 17 [21] and Spring Boot [22]: The core business logic is developed using Java 17, taking advantage of its modern features and performance optimizations. Spring Boot was chosen as the foundational framework due to its ability to rapidly bootstrap

lightweight, standalone microservices with embedded servers. It significantly reduces configuration overhead.

RESTful API Architecture [23]: The microservices expose their business capabilities via standard RESTful web services. This stateless, client-server architectural style ensures that the frontend and external services can interact with the backend in a standardized, predictable manner.

MySQL [24] and MySQL Workbench [25]: We chose MySQL as our primary Relational Database Management System (RDBMS) due to its reliability and robustness. MySQL Workbench was utilized as our visual tool for database design, schema management, and administration.

Spring Data JPA (ORM) [26]: For data persistence, we integrated Spring Data JPA. This Object-Relational Mapping (ORM) layer seamlessly bridges the gap between our object-oriented domain model and the underlying MySQL database, eliminating boilerplate SQL code.

Spring Cloud OpenFeign [27]: To enable synchronous inter-service REST communication (e.g., the Leave Management service verifying user roles with the User Management service), we utilized OpenFeign as a declarative web service client.

Swagger [28] / OpenAPI Specification [29]: We integrated Swagger UI to document, visualize, and test REST APIs. In this project, the OpenAPI specification acts as a machine-readable contract used by our multi-agent framework to understand system interfaces and generate executable tests. It is also used as an input source for the automated test system.

3.2.2.2 Tools used for Test System

LLM Orchestration and Agent Frameworks

The orchestration layer is responsible for coordinating the execution flow of multiple agents and managing state propagation across the pipeline.

LangGraph [30] is used as the core orchestration framework. It models the system as a directed graph where each node represents an agent and edges define transitions. Its native support for shared state and cyclic execution makes it particularly suitable for feedback-driven workflows.

LangChain [31] is used as a complementary abstraction layer for prompt engineering and LLM interaction. It provides structured components for prompt templates and model invocation within individual agents.

These choices are the result of a comparative study and extensive research conducted during the design phase, which demonstrated that LangGraph and LangChain are the most suitable frameworks for meeting the requirements and complexity of our multi-agent system.

Model Access and Context Augmentation

This layer handles interaction with large language models and external knowledge sources.

Groq [32] is a high-performance inference platform for large language models, optimized for ultra-low latency and consistent responses via specialized hardware. In this project, it is used as the main LLM provider to replace traditional API-based solutions and improve pipeline speed and efficiency.

Several options were evaluated. Ollama was limited by local hardware constraints, while Hugging Face free models imposed parameter restrictions. Groq offered the best balance between performance and model capacity without hardware dependency, and was therefore selected as the final solution

ChromaDB [33] is used as a vector database to support Retrieval-Augmented Generation (RAG). It enables semantic retrieval of relevant context, which is injected into prompts to improve output quality and domain alignment.

Test Execution and Validation Stack

This layer is responsible for executing and validating automated tests.

Cucumber (JVM) [34] is used as the Behavior-Driven Development (BDD) engine for executing Gherkin scenarios.

JUnit 5 [35] serves as the underlying test execution framework integrated with Cucumber.

RestAssured [36] is used for API testing, providing a fluent interface for HTTP request validation.

Maven [37] is used for build automation and dependency management, enabling execution of the test pipeline through standard lifecycle commands.

JaCoCo [38] is used for code coverage analysis, providing runtime metrics on test effectiveness.

Data Validation and Configuration

This layer ensures consistency and correctness of structured data exchanged between agents.

Pydantic [39] is used to validate and enforce structured outputs generated by language models.

PyYAML [40] is used to parse declarative business requirement files defined in YAML format.

Frontend Layer

This layer is responsible for the user interface and interaction with the test system.

React is used as the frontend framework, following a component-based architecture that enables efficient rendering of dynamic views. The application is built with **React**

19, which introduces enhanced support for server actions and simplified asynchronous state management, improving both performance and developer experience. Core React hooks (`useState`, `useEffect`, and `useMemo`) are used throughout the application, with `StrictMode` enabled for development safety.

Vite serves as the build tool, providing fast development server startup and optimized production builds through standard lifecycle commands (`npm run dev`, `vite build`, and `vite preview`).

Tailwind CSS is used for styling. It provides a utility-first approach for building a custom visual theme, including gradient backgrounds, glassmorphism panels, and workflow animations.

3.2.3 RAG Integration

Building on the RAG architecture introduced in Chapter 2, this section details its concrete implementation within the prototype, covering the corpus design, ingestion pipeline, retrieval configuration, and the specific way each agent consumes the retrieved context.

RAG Type

The system adopts a **Standard RAG** approach (also referred to as Naive RAG in the literature), following a straightforward *ingest* $\rightarrow$ *embed* $\rightarrow$ *retrieve* $\rightarrow$ *inject* pipeline. The ingestion phase is performed **offline**, prior to any agent execution: documents are chunked, embedded, and persisted in a local vector store. The retrieval phase is **online**, triggered at agent runtime whenever additional domain context is needed. No query transformation, re-ranking, or multi-hop retrieval is applied; the pipeline relies solely on dense cosine-similarity search over the indexed chunks.

As illustrated in Figure 3.2, the pipeline is structured into three distinct phases: offline data ingestion, vector storage via ChromaDB, and online retrieval consumed by the Scenario Designer and Gherkin Generator agents.

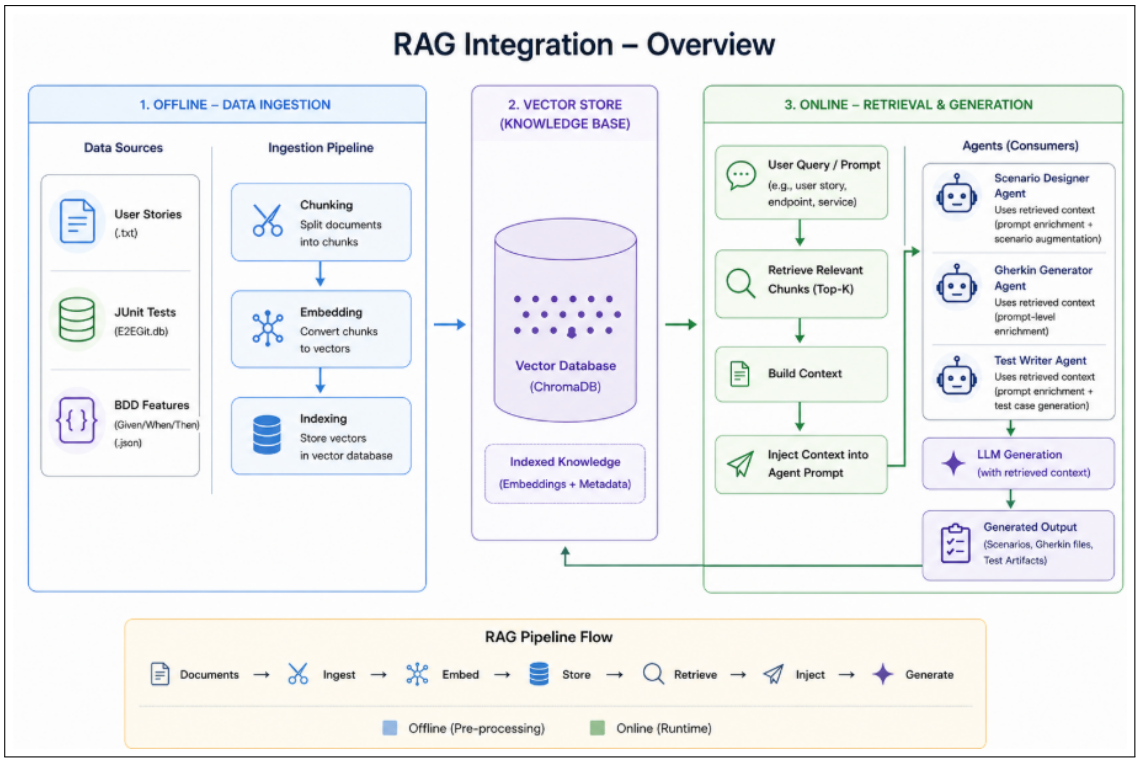


Figure 3.2: RAG Integration Overview

Knowledge Base and Datasets

The knowledge base is built from three heterogeneous corpora, each loaded by a dedicated parser in `ingest.py` and stored in the same ChromaDB collection. Table 3.1 summarises the three sources.

Table 3.1: RAG Knowledge Base Datasets

Dataset	Content	Format
`z13880060_user_stories`	Real-world user stories	Plain text (.txt)
`z14234731_e2egit` (E2EGit.db)	GUI Java JUnit test cases	SQLite → CSV
`GivenWhenThen.json`	BDD feature files & step definitions	JSON

Each loader extracts the most informative text fields: story text for the first corpus, concatenated test content for the second, and a combination of feature content, step definitions, and system code files for the third. All documents are tagged with source metadata (`dataset`, `source_table`, `type`) to preserve traceability after retrieval.

Ingestion Pipeline

Once loaded, all documents pass through the same ingestion chain implemented in `ingest.py`:

1. **Chunking.** Documents are split using `RecursiveCharacterTextSplitter` with a chunk size of **1200 characters** and an overlap of **150 characters**, using the sep-

arator hierarchy `[\n\n, \n, space, empty]`. This strategy preserves paragraph boundaries while keeping chunks small enough for precise retrieval.

2. **Deduplication.** Each chunk is assigned a stable identifier computed as the **SHA-1 hash** of its content concatenated with its serialized metadata. A secondary suffix mechanism handles the rare case of identical hashes within the same upsert batch, ensuring uniqueness across all IDs submitted in a single Chroma upsert call.

3. **Embedding.** Chunks are encoded using the `sentence-transformers/all-MiniLM-L6-v2` model loaded via `HuggingFaceEmbeddings`[41]. This lightweight model runs locally without any API call, producing dense vector representations suitable for semantic retrieval.

4. **Indexing.** Embeddings are persisted in a ChromaDB vector store (collection `tier3_rag`, persist directory `chroma_db/`), written in batches of **256 chunks** to limit peak memory usage.

Retrieval Configuration

At runtime, the `retriever.py` module exposes a LangChain-compatible retriever built on top of the persisted Chroma collection. The retrieval parameters are summarised in Table 3.2.

Table 3.2: RAG Retrieval Configuration

Parameter	**Value**
Embedding model	`sentence-transformers/all-MiniLM-L6-v2`
Vector store	ChromaDB (`tier3_rag`)
Chunk size / overlap	1200 / 150 characters
Similarity metric	Cosine similarity (Chroma default)
Top-K chunks retrieved	5 (configurable via `RAG_TOP_K`)
Max injected context	2500 characters (`RAG_MAX_CHARS`)
Re-ranking	None
Ingestion mode	Offline (pre-runtime)
Retrieval mode	Online (agent runtime)

Knowledge Base

The RAG knowledge base is assembled from three source datasets stored under `data/raw/`:

- `GivenWhenThen.json` — over 55,000 real-world BDD scenarios drawn from open-source repositories, covering Java, Ruby, and Node.js implementations alongside their step definitions and system code files.
- `E2EGit.db` — approximately 5,000 Selenium GUI test patterns sourced from end-to-end test repositories.

- User story text files — business requirements used as domain context during retrieval.

All three sources are processed through the same ingestion pipeline described above and persisted in the `tier3_rag` Chroma collection.

Agent-Level Integration

The RAG component is consumed differently by each agent, reflecting their distinct generation objectives. Exactly three agents query the retriever directly: the Scenario Designer Agent, the Gherkin Generator Agent, and the Test Writer Agent. In all cases, RAG retrieval is **offline and local** — the vector store is a persisted Chroma database queried via local embeddings, with no web search involved. LLM calls, by contrast, are **online**, routed to the Groq API through a shared model factory.

Scenario Designer Agent. The Scenario Designer Agent integrates RAG at **two levels**:

- **Prompt-level enrichment.** Before each LLM call, the agent queries the index with a representation of the current service and its endpoint catalog. The retrieved chunks are formatted as a concrete examples block and appended to the LLM prompt, giving the model real-world BDD patterns to draw from.
- **Output-level augmentation.** After the LLM returns its scenarios, the agent issues four fixed targeted queries: invalid input validation errors, boundary value edge cases, unauthorized access security patterns, and null or empty payload rejection. Irrelevant results are filtered out (rejecting foreign protocols such as SMTP, Kafka, and Redis, and requiring a keyword overlap of at least 2 with the service endpoints). Remaining chunks are classified into `happy_path`, `error_case`, `edge_case`, or `security`, and mapped back onto the real API endpoints before being converted into scenario objects. These are appended to the generated scenario list **without passing through the LLM**, making this mechanism particularly effective for injecting error, edge, and security scenarios sourced from the GivenWhenThen corpus.

This dual-level usage means that RAG influences the Scenario Designer Agent both during generation (via prompt context) and after generation (via direct scenario injection), maximising coverage without inflating the LLM prompt size. The agent is therefore **hybrid**: retrieval is local, while scenario generation relies on an online LLM call.

Gherkin Generator Agent. The Gherkin Generator Agent integrates RAG at the **prompt level only**. The query is the raw user story block currently being processed. Up to 5 chunks are retrieved, trimmed to a maximum of 2500 characters, and injected as

the first input in the LLM prompt — clearly labelled and positioned before the Swagger contract to avoid confusion between domain context and API facts. Swagger remains the explicit source of truth; RAG provides supporting domain context only, not contract authority. Any retrieval failure is logged and silently ignored, ensuring that the RAG layer never blocks the generation pipeline. Like the Scenario Designer Agent, the Gherkin Generator Agent is **hybrid**: local RAG retrieval feeds an online LLM call for final Gherkin generation.

Test Writer Agent. The Test Writer Agent integrates RAG at the **step level**, making its usage lighter and more targeted than the other two agents. For each Gherkin step being processed, the agent queries the retriever with the step text and inserts the retrieved context as a comment block directly above the generated Java step method. Importantly, the retrieved chunks do **not** control the method body logic or endpoint selection — the deterministic generation path remains responsible for those decisions. RAG therefore serves primarily as **documentation and traceability enrichment**, embedding realistic prior examples as inline comments in the generated test code. This path attempts retrieval and silently skips on failure without any explicit guard.

Regarding LLM usage, the Test Writer Agent follows a **deterministic-first** strategy: step methods are generated locally without an LLM call in the normal path. An online LLM is invoked only in fallback generation, failure-guided repair, and consolidated repair modes. Consequently, the Test Writer Agent's runtime profile is predominantly **local** — both its RAG retrieval and its primary code generation path operate offline, with online LLM calls reserved for exception and repair scenarios.

Agents Without Direct RAG Access. The Gherkin Validator Agent, Test Executor Agent, Failure Analyst Agent, and Coverage Analyst Agent do not query the retriever directly. The Gherkin Validator Agent uses an online LLM for semantic validation but has no RAG integration. The Failure Analyst Agent and Coverage Analyst Agent are fully offline, performing local analysis of Cucumber JSON and JaCoCo reports respectively. The Test Executor Agent is likewise local, invoking Maven and the running backend services without any retrieval or LLM calls.

3.3 Graphical User Interface (GUI)

In this section, we present a selection of screenshots of the graphical interfaces developed in our platform.

VORTEX Dashboard

The dashboard displays the orchestration mode, enabled services, and the current pipeline status which can be idle or running. A workflow tracker shows the progression of multiple agents, including Scenario Design, Gherkin Generator, Validation, Test Writer, and

others. Initially, no agent has been executed.

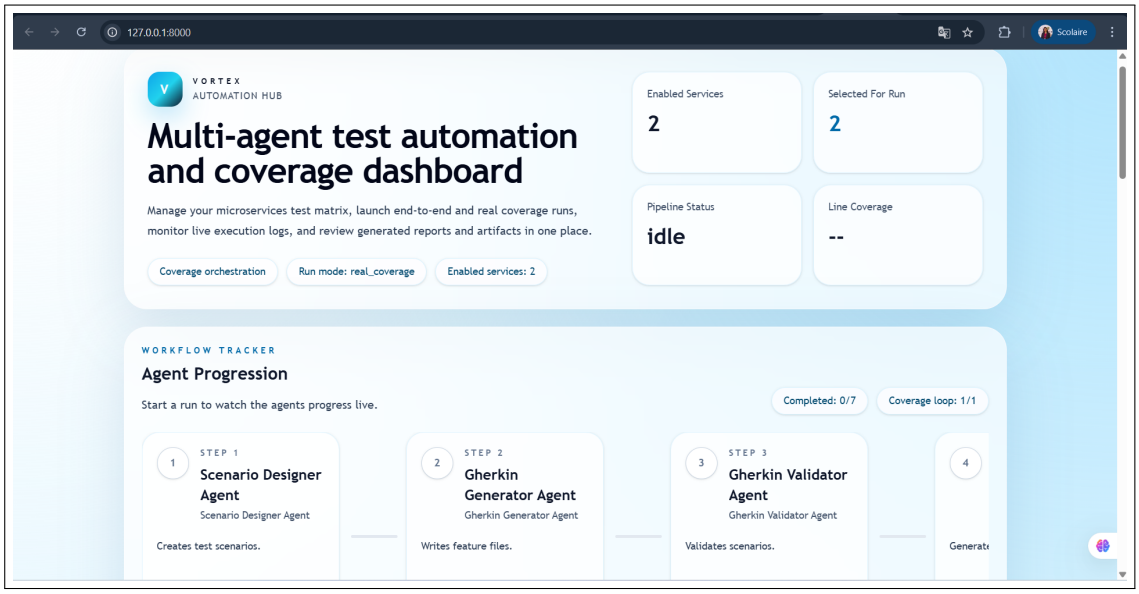


Figure 3.3: VORTEX dashboard with coverage orchestration and agent workflow tracker.

Microservices Configuration View

This interface allows users to manage individual microservices and their dependencies. Top action buttons provide options to add a new service, save inputs, or execute tests directly. Each service (e.g., auth, leave) can be enabled or removed, with configurable fields such as port, base URL, Swagger file location, and role. The bottom panel provides global run settings including execution mode, selected services, timeout, output path, and JWT authentication credentials.

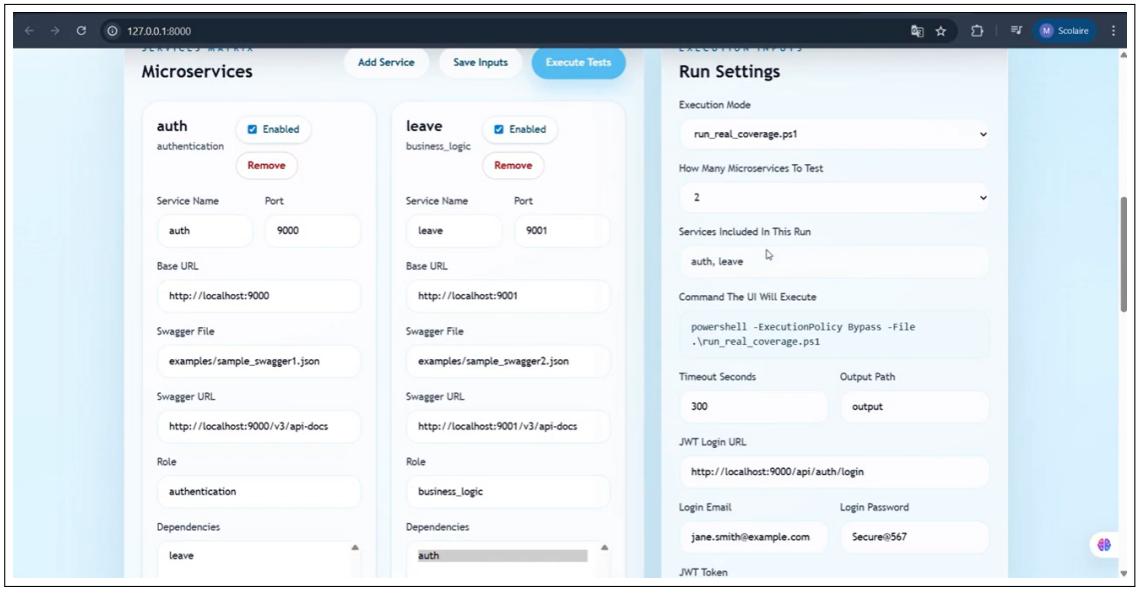


Figure 3.4: Microservices configuration and run settings panel.

Requirements and Execution Results View

The left panel contains two input document blocks: User Stories (with leave request lifecycle, validation, approval, and boundary tests) and Business Requirements YAML (defining services, user stories, business rules, endpoints, test scenarios, and integration scenarios). The right panel displays execution results including bundle details, generation timestamp, failures, quality gate status, coverage metrics (line, branch, method), and artifacts.

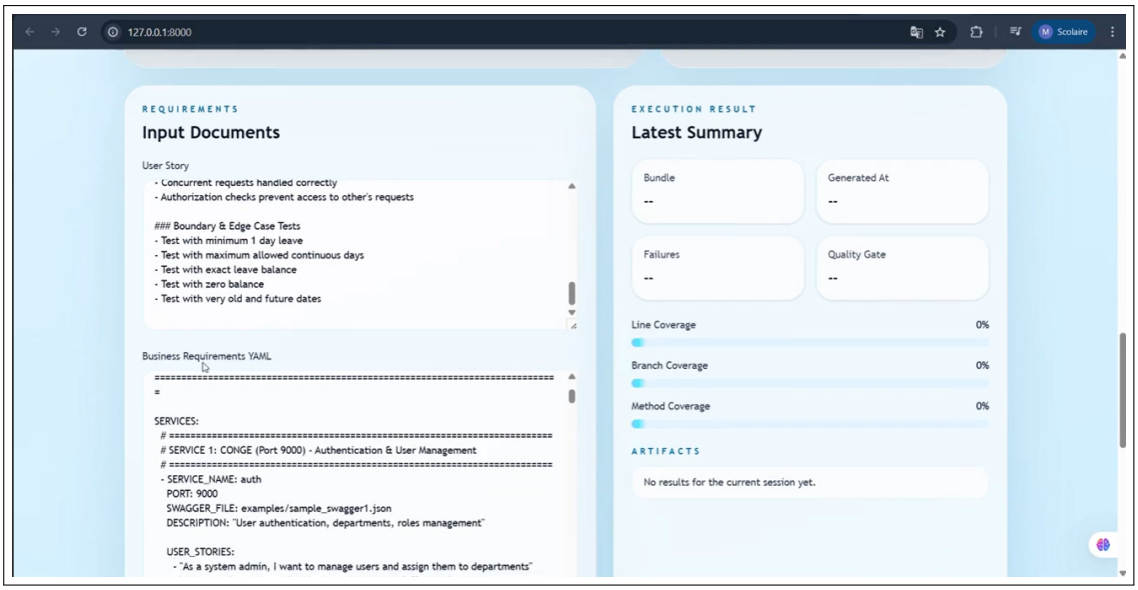


Figure 3.5: Requirements input with user stories, business rules, and execution results panel.

Agent Progression During Test Execution

Once the user clicks the Execute Tests button, the pipeline status changes to running and agents begin processing. The workflow tracker shows the Validation and Test Writing steps as completed, Test Execution as currently active, while Execution and Coverage steps remain pending. The Services Matrix panel on the right confirms the running state.

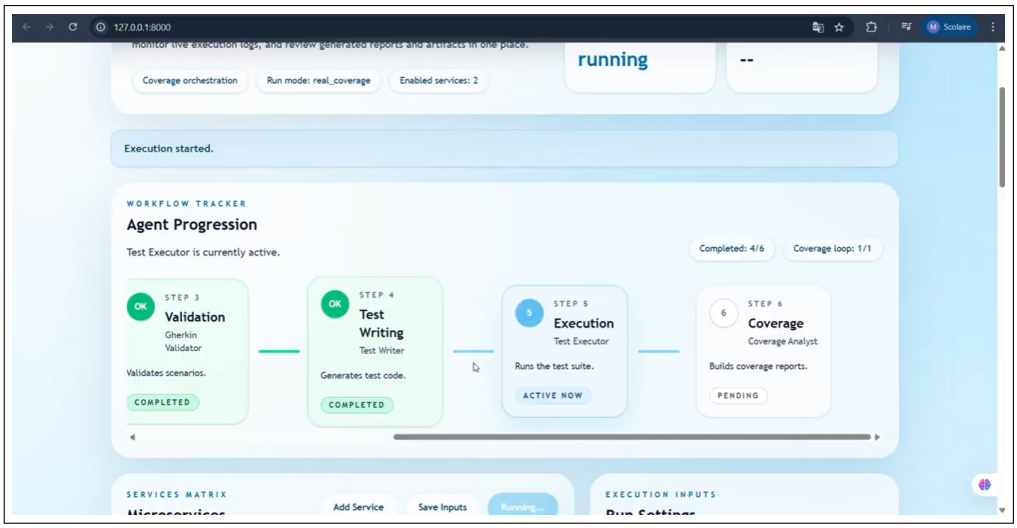


Figure 3.6: Agent progression workflow with completed, active, and pending steps during test execution.

Pipeline Console Logs

This panel displays real-time execution logs from the test pipeline. While the console can show logs from all agents, the current capture specifically shows INFO and WARNING messages generated by the test executor agent, including feature file paths, working directories, Maven commands with service base URLs, and timestamp warnings about expired tokens.

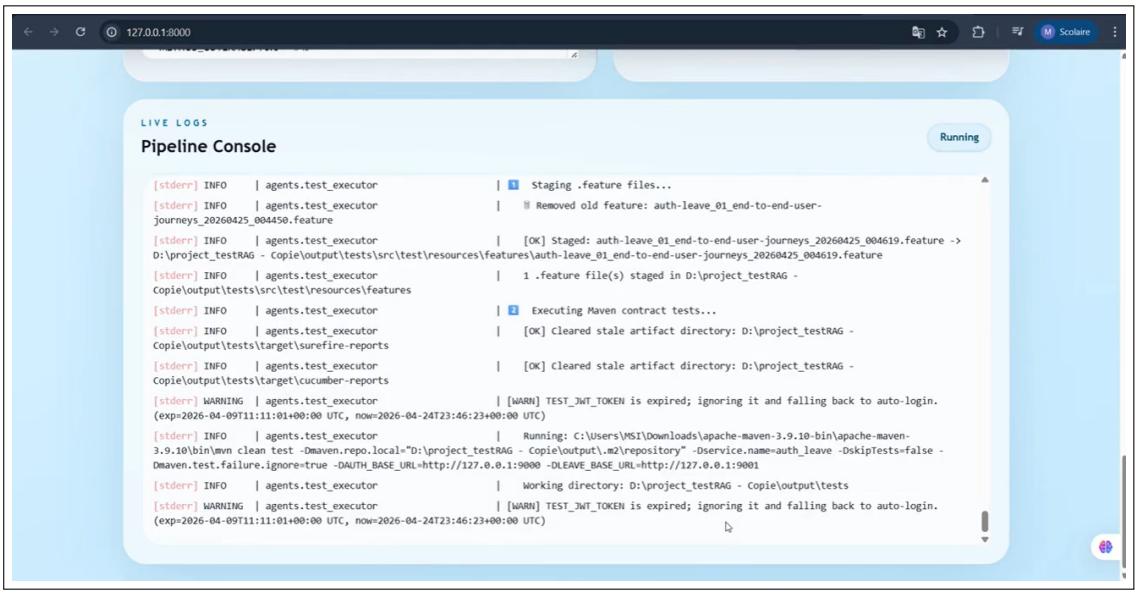


Figure 3.7: Pipeline console showing real-time execution logs from the test executor agent.

Execution Results with Coverage Metrics and Artifacts

The execution result panel displays the latest test summary including bundle name, generation timestamp, number of failures, and quality gate status. Coverage metrics show line coverage, branch coverage, and method coverage percentages. The artifacts section lists

generated output files such as coverage reports, Cucumber HTML reports, verification reports, and feature files.

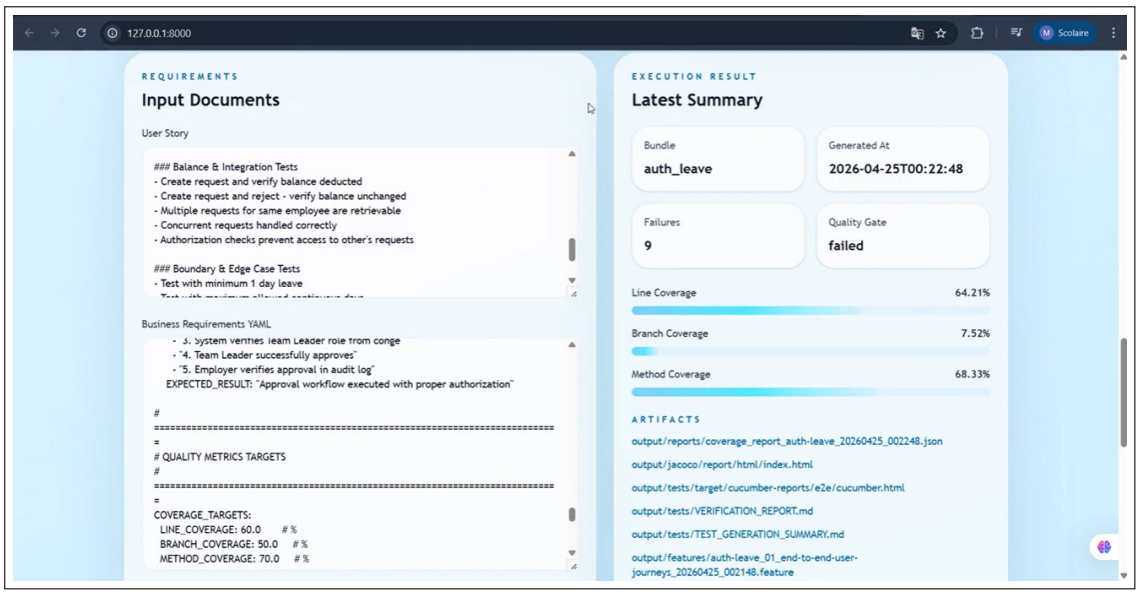


Figure 3.8: Execution results with coverage metrics, quality gate, and generated artifacts.

Test Report Artifact (Cucumber HTML Report)

By clicking on the Cucumber HTML report link in the artifacts section which is located in the execution results block, the user can view detailed test execution results. The report shows which scenarios passed or failed, including security test cases (e.g., AUT-0175, AUT-0185, LEA-0215). For each failure, the report displays the expected outcome, the actual response received (e.g., 201 instead of 403), and the corresponding assertion error stack trace.

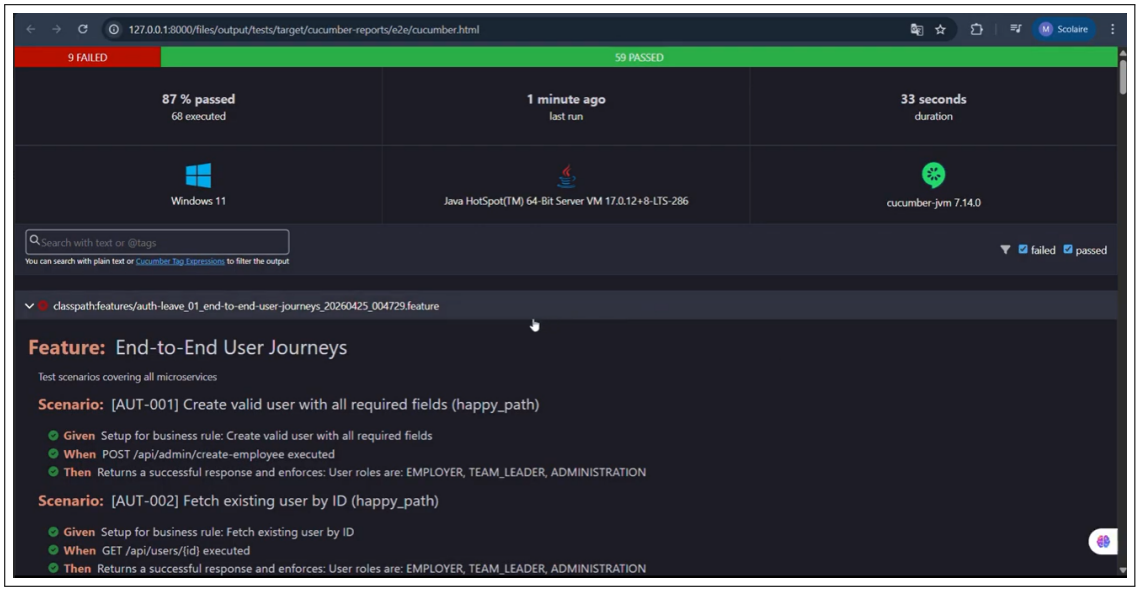


Figure 3.9: Cucumber HTML report showing passed and failed test scenarios.



Figure 3.10: Detailed error view with assertion failures and stack traces for security scenarios.

3.4 Experimental Setup and Evaluation

This section describes the experimental methodology used to evaluate the multi-agent test generation pipeline. We define the evaluation criteria, detail the per-agent LLM selection process based on empirical benchmarks, and present a comparative performance analysis across three architectural configurations.

Metric Definitions and Pipeline Funnel

The seven evaluation metrics form a sequential conversion funnel: each stage can only produce meaningful output if the prior stage succeeds.

Table 3.3: Evaluation metrics and their role in the pipeline funnel.

Metric	Full Name	Description and Funnel Role	Range
SC	Scenario Coverage	First gate: % of reference scenarios successfully generated. A low value limits overall coverage regardless of downstream performance.	0–100%
SV	Syntax Validity	Second gate: % of generated scenarios passing lint validation. Filters invalid Gherkin before compilation.	0–100%
TC	Test Compilation	Third gate: 100% if Maven compilation succeeds, 0% otherwise. A 0% value halts the entire downstream evaluation.	0% or 100%
ESR	Execution Success Rate	% of executed scenarios that completed without runtime exceptions. Distinguishes crashes from assertion failures.	0–100%
TPR	Test Pass Rate	% of executed scenarios where all assertions passed. Primary business-level correctness indicator.	0–100%
RSR	Repair Success Rate	100% if the self-healing workflow completed at least one successful repair. Measures value added beyond initial generation.	null, 0% or 100%
GT	Generation Time	Total end-to-end duration of the full pipeline run. Measures whether the pipeline fits within CI/CD time budgets.	≥ 0 s

Key Insight. SC is the dominant bottleneck across all experiments. Even when all downstream metrics reach 100%, insufficient scenario generation fundamentally limits overall test coverage, making it the highest-leverage optimisation target.

3.4.1 Per-Agent LLM Selection

To identify the most suitable LLM for the `GherkinGeneratorAgent` — the first and most critical agent in the pipeline — we conducted an isolated benchmark . Each model was given a fixed user story and evaluated on both generation quality and token efficiency.

3.4.1.1 Quality Benchmark

Figure 3.11 compares three candidate models across four quality dimensions: generation duration, scenario count, step count, and `gherkin-lint` issues.

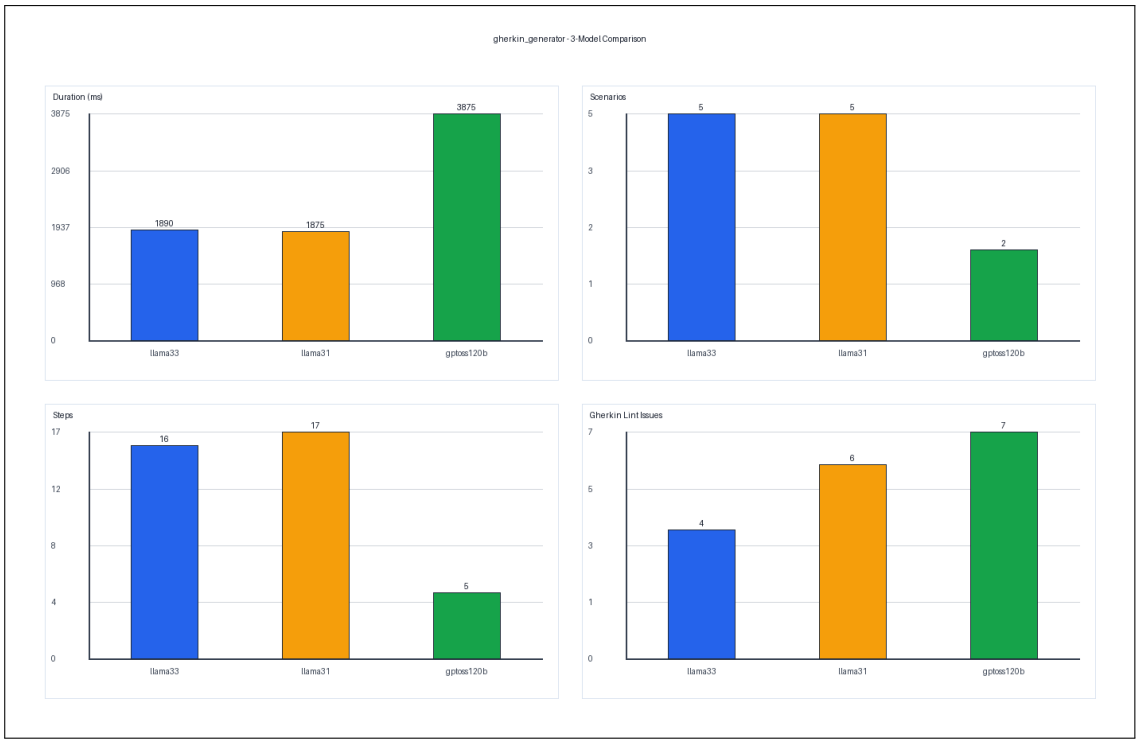


Figure 3.11: Gherkin GeneratorAgent quality benchmark across three LLM candidates.

Table 3.4: Numerical results extracted from the quality benchmark run.

Model	Run Time (ms)	Scenarios	Steps	Lint Issues	Total Tokens
llama33	1,890	5	16	4	3,002
llama31	1,875	5	17	6	2,994
gptoss120b	3,875	2	5	7	4,750

The results reveal a clear performance hierarchy:

- **llama33** (`llama-3.3-70b-versatile`): 5 scenarios, 16 steps, 4 lint issues at 1,890 ms — best balance of quality and lint compliance.
- **llama31** (`llama-3.1-8b-instant`): 5 scenarios, 17 steps, 6 lint issues at 1,875 ms — competitive speed, but slightly higher lint errors.
- **gptoss120b**: only 2 scenarios with the highest lint issue count (7) despite the longest runtime (3,875 ms) — worst quality/cost ratio.

3.4.1.2 Token Efficiency Benchmark

Figure 3.12 provides the corresponding token consumption analysis, which directly impacts API cost and latency at scale.

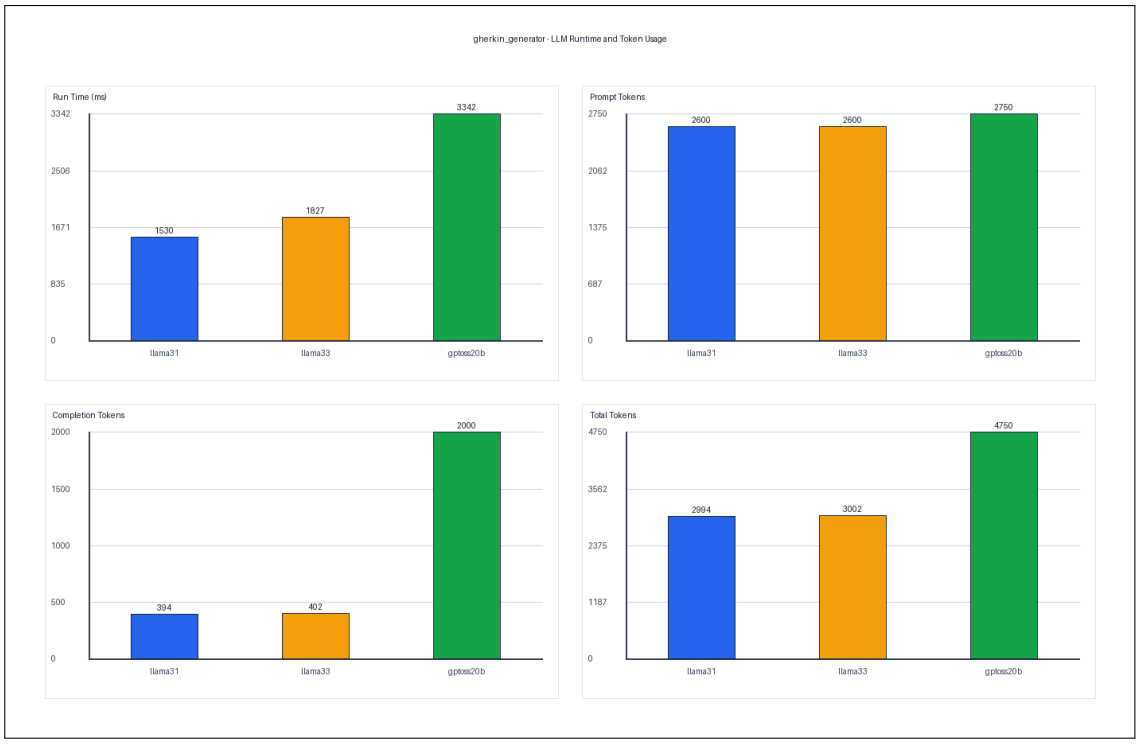


Figure 3.12: Gherkin Generator Agent token usage benchmark.

The token breakdown confirms the earlier quality findings from a cost perspective:

- **llama31** and **llama33** consume approximately 3,000 total tokens each, producing similar scenario counts — a near-identical cost/quality trade-off.
- **gptoss120b** consumes 4,750 total tokens (58% more than the Llama models) and generates 2,000 completion tokens, yet delivers substantially fewer and lower-quality scenarios.

Selection Rationale. Based on this benchmark, **llama33** (`llama-3.3-70b-versatile`) was selected as the default LLM for the `Gherkin Generator Agent`. It achieves the best quality metrics (fewest lint issues, comparable scenario count) at a competitive token cost. **gptoss120b** is explicitly excluded from production configurations due to its poor efficiency profile.

3.4.2 Performance Analysis Across Versions

We evaluate the pipeline across three progressive architectural configurations to quantify the contribution of each design decision:

- **Baseline (no RAG, no feedback loop):** direct LLM generation with no retrieval augmentation and no iterative refinement.

- **+ RAG only:** the most relevant chunks are retrieved from the local knowledge base and injected into the prompt context, enriching the generator with real-world BDD examples. Swagger documentation is not the source of retrieval here.
- **+ RAG + Feedback Loop:** a single post-execution feedback iteration analyses failed tests and rewrites the responsible scenarios, on top of the RAG-enriched generation.

3.4.2.1 Impact of the Feedback Loop

Figure 3.13 presents a direct comparison between the baseline single-pass configuration (*Without Loop*) and the single-iteration feedback variant (*One Loop*), using the **llama33** model throughout.

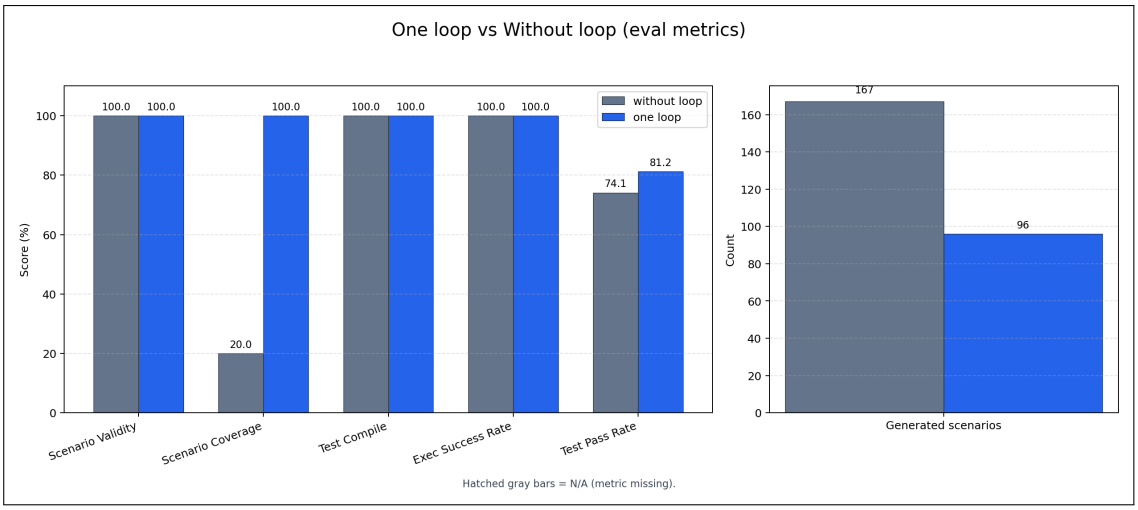


Figure 3.13: Evaluation metrics and scenario count: One Loop vs. Without Loop.

Table 3.5: Numerical comparison of pipeline variants with and without the feedback loop.

Condition	SV	SC	TC	ESR	TPR	Scenarios
Without Loop	100%	20%	100%	100%	74.1%	167
One Loop	100%	100%	100%	100%	81.2%	96

The results demonstrate clear and consistent improvements from the feedback loop:

- **Scenario Coverage (SC):** increases from 20% to 100%, representing the most significant gain. The loop triggers the Scenario Designer Agent to generate missing cases identified during execution, transforming partial coverage into complete business-requirement alignment.

- **Test Pass Rate (TPR):** improves from 74.1% to 81.2% (+7.1 pp), confirming that the failure analyst successfully repairs a meaningful fraction of broken test assertions.
- **Scenario count:** decreases from 167 to 96. This apparent reduction is a consolidation effect — the loop eliminates redundant or duplicate scenarios while increasing meaningful coverage, producing a more compact and higher-quality test suite.
- **SV, TC, and ESR:** remain at 100% across both configurations, demonstrating that the feedback loop preserves syntactic and execution correctness while improving semantic quality.

3.4.2.2 Summary of Version Comparison

Table 3.6 synthesises the key metrics across all three pipeline configurations, as recorded in the evaluation output files.

Table 3.6: Performance comparison across all three pipeline versions.

Configuration	SV	SC	TC	ESR	TPR
Baseline (no RAG, no loop)	100%	5%	100%	100%	32%
+ RAG only	100%	17.8%	100%	100%	76%
+ RAG + Feedback Loop	100%	100%	100%	100%	81.2%

The results reveal a consistent upward trend across all three configurations: RAG alone has a moderate impact on SC (+12.8 pp) and a strong effect on TPR (+44 pp), by grounding generated tests in real-world examples that better reflect the expected system behaviour. Adding the feedback loop resolves the remaining SC gap completely, achieving full requirement coverage while further improving TPR.

Key Insight. The combination of RAG and a single feedback iteration transforms the pipeline from a low-coverage prototype (5% SC, 32% TPR) into a production-viable system (100% SC, 81.2% TPR). Each architectural addition makes a measurable, non-redundant contribution to overall pipeline quality.

3.4.2.3 Code Coverage Analysis: Loop vs. No Loop

Beyond the pipeline-level metrics reported above, we examine the **JaCoCo code coverage** collected during test execution for both configurations. These metrics — line, branch, and method coverage — provide a complementary, implementation-level view of how thoroughly the generated tests exercise the System Under Test. The values were extracted directly from the pipeline execution logs produced by the Coverage Analyst Agent .

Figure 3.14 presents the grouped bar chart comparing the three coverage dimensions across both configurations.

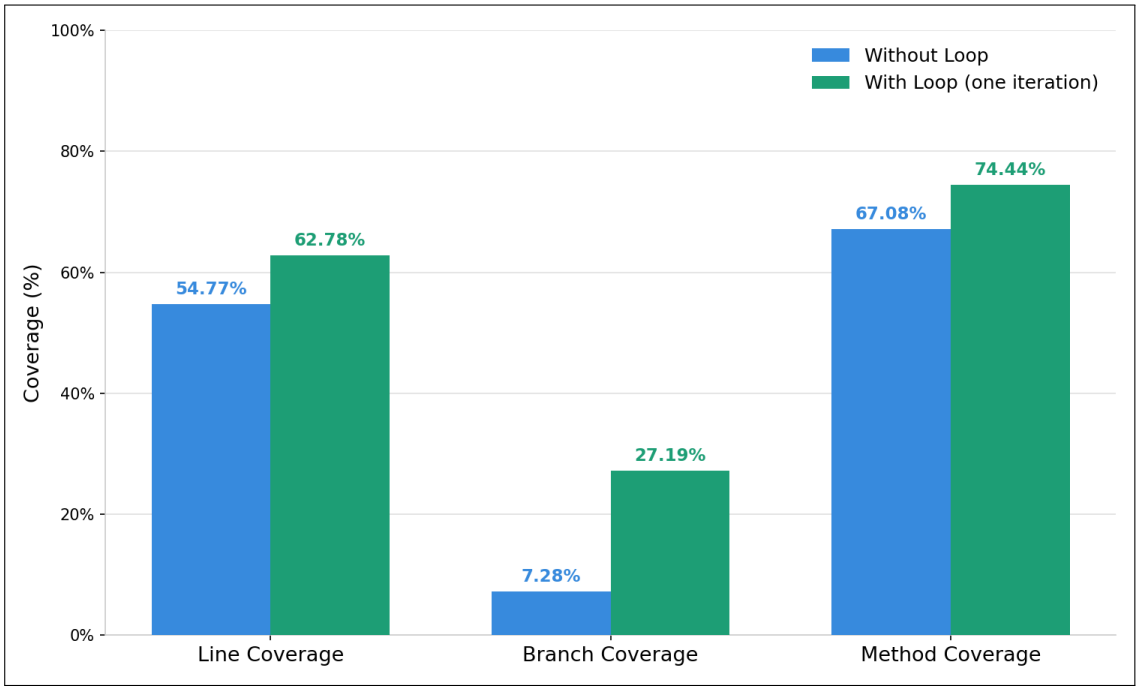


Figure 3.14: JaCoCo code coverage comparison: Without Loop vs. One Loop iteration.

Table 3.7: JaCoCo coverage metrics extracted from pipeline execution logs.

Configuration	Line Coverage	Branch Coverage	Method Coverage
Without Loop	54.77%	7.28%	67.08%
With Loop	62.78%	27.19%	74.44%

The results reveal a nuanced trade-off between the two configurations:

- **Branch Coverage** shows the most significant gain with the feedback loop, rising from 7.28% to 27.19% (+19.91 pp). This is the most meaningful improvement from a quality assurance perspective: branch coverage measures whether both the true and false outcomes of every conditional statement in the source code are exercised. The without-loop configuration generates a larger but less targeted set of scenarios (167 scenarios), which exercise many lines sequentially without probing conditional branches. The self-healing loop, by contrast, regenerates failing scenarios with corrected logic, producing tests that reach deeper into conditional paths.
- **Method Coverage** also improves with the loop, increasing from 67.08% to 74.44% (+7.36 pp), indicating that the consolidated, higher-quality test suite exercises a broader range of service methods despite containing fewer total scenarios.

- **Line Coverage** decreases slightly from 62.78% to 54.77% (−8.01 pp). This apparent regression is consistent with the consolidation effect observed in Table 3.5: the feedback loop reduces the total scenario count from 167 to 96 by removing redundant duplicates. Fewer scenarios naturally execute fewer lines in aggregate; however, the substantial gains in branch and method coverage confirm that the retained scenarios are of higher individual quality.

Key Insight. The feedback loop shifts the coverage profile from *broad but shallow* (high line coverage, negligible branch coverage) to *focused and deep* (stronger branch and method coverage). This confirms that the self-healing mechanism does not simply add more tests — it generates *better* tests that probe the conditional logic of the system more effectively.

3.4.3 Comparative Analysis: Expert vs. LLM-Generated Scenarios

In this section, we conduct a comparative analysis of two Gherkin specifications for the 'Approve Leave Request' user story. We contrast a baseline scenario authored by a domain expert with a version generated via our proposed LLM-based methodology. This comparison evaluates the effectiveness and structural quality of our approach against human-generated standards.

3.4.3.1 Domain Expert Gherkin Specification for Leave Request Approval Process

The following scenarios define the validation workflow of leave requests. They cover final approval, intermediate approval, and invalid validation cases, ensuring correctness, traceability, and role-based control.

Gherkin Scenarios Generated by a Domain Expert

Scenario 1: Final Validation by Last Approver
Given: User (Administrator) authenticated; request is *Pending/In Progress*; user is last approver.
When: User views request, optionally adds observation, submits approval.
Then: System verifies chain membership and prior validation; updates balance; sets status *APPROVED*; stores validation data; displays success message.

Scenario 2: Intermediate Validation
Given: User authenticated; request is *Pending/In Progress*; user is not last approver.
When: User views request, optionally adds observation, submits approval.
Then: System marks validation TRUE; keeps status *In Progress*; stores observation; displays success message.

Scenario 3: Invalid Status or Rights
Given: User authenticated; request already Approved/Canceled/Refused or invalid workflow.
When: User attempts approval.
Then: System rejects request and returns error message.

Scenario 4: Duplicate Validation
Given: User authenticated; request already validated by this user.
When: User attempts validation again.
Then: System blocks action and shows "Validation already performed".

3.4.3.2 Automated Test Scenarios for Leave Request Approval Endpoint

Description: The following scenarios are automatically generated to validate the robustness of the API endpoint `/api/leave-requests/{id}/approve`. They cover functional behavior, error handling, security constraints, boundary conditions, and end-to-end workflow validation.

Gherkin Scenarios Generated by Our Solution

Scenario: [LEA-003] Valid PUT (happy_path)
Given: valid parameters for the endpoint.
When: PUT request sent with valid data.
Then: system returns success response (200/201).

Scenario: [LEA-003E] Missing Required Fields
Given: required parameters are missing.
When: PUT request sent without required fields.
Then: system returns **400 Bad Request**.
Scenario: [LEA-003S] Unauthorized Access
Given: no authentication token provided.
When: PUT request sent without JWT token.
Then: system returns **401 Unauthorized**.

Scenario: [LEA-003N] Non-Existent Resource
Given: resource does not exist.
When: PUT request sent with non-existent ID.
Then: system returns **404 Not Found**.

Scenario: [LEA-003X] Boundary Values
Given: boundary values (0, empty string, maximum integer).
When: PUT request sent using boundary values.
Then: system handles input gracefully or rejects with clear error.
Scenario: [E2E-001] Authentication → Leave Flow
Given: user creates leave request.
When: team leader approves leave.
Then: leave request successfully approved.

Scenario: [E2E-003] Role-Based Approval Workflow
Given: employee creates leave request.
When: team leader approves after role verification.
Then: workflow executed with proper authorization.

3.4.3.3 Comparative Analysis

Objective: This section compares expert-written Gherkin scenarios (business-oriented) with automatically generated scenarios (technical-oriented), focusing on coverage, precision, and testing effectiveness.

1. **Scope and Intent**

- **Expert Gherkin:** Business logic and workflow validation (roles, approval chain, request states).

- **Generated Gherkin:** API-level validation (HTTP responses, security, input handling).

Insight: Expert scenarios define expected behavior, while generated scenarios validate system execution at API level.

2. Coverage

- **Expert:** Strong on business rules and workflow consistency; detects logical errors.
- **Generated:** Strong on technical behavior (status codes, security, edge cases).

Limitation: Expert lacks technical validation; generated lacks deep business rules.

3. Precision and Maintainability

- **Expert:** Precise, stable, aligned with real processes.
- **Generated:** More abstract and sensitive to implementation changes.

4. Risk Coverage

Risk	Expert	Generated
Business logic	High	Low
API failures	Low	High
Security	Low	High
Workflow issues	High	Low
Input validation	Medium	High

Conclusion: Both approaches are complementary: expert scenarios ensure business correctness, while generated scenarios ensure technical robustness. A complete testing strategy requires both perspectives.

3.5 Conclusion

This chapter presented the implementation and evaluation of the VORTEX multi-agent framework across three architectural configurations: baseline LLM generation, RAG integration, and a self-healing feedback loop. Each stage improved test quality, with RAG grounding outputs in the API contract and increasing passing assertions, while the feedback loop ensured full alignment with business requirements.

A comparative analysis with expert-written scenarios showed that both human and automated approaches are complementary. Overall, the results validate VORTEX's design and its suitability for automated functional testing in microservice architectures.

General Conclusion

This project led to the design and implementation of VORTEX, an automated functional test generation framework built on a multi-agent architecture driven by large language models, addressing a concrete industrial challenge in BDD-based software quality assurance: how to automatically produce test scenarios simultaneously aligned with business requirements and technically validated against the actual system behaviour.

The main contributions of this work are:

- A multi-agent pipeline automating the full BDD testing lifecycle, from Gherkin scenario generation to test execution and self-healing.
- A systematic evaluation of large language models for Gherkin generation, identifying the most suitable candidate in terms of output quality and token efficiency.
- The integration of RAG to anchor generated tests to the real API contract, substantially improving test correctness.
- A self-healing feedback loop that iteratively consolidates the test suite, achieving higher pass rates while eliminating redundant scenarios.

The three chapters progressively established the theoretical foundations, detailed the technical blueprint of the seven-agent pipeline, and validated the framework through controlled experiments. Results confirmed that RAG and the feedback loop are two independent, non-redundant levers that together transform a limited prototype into a production-ready system, demonstrating that RAG-augmented multi-agent systems constitute an effective means of modernising software quality assurance.

Looking ahead, open directions include improving coverage through richer RAG corpora, extending the framework to event-driven architectures, and validating scalability across larger microservice deployments, pointing toward a fully automated, domain-agnostic testing assistant integrated into modern CI/CD pipelines.

Bibliography

[1] NVIDIA, "What is retrieval-augmented generation (rag)?." https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/, 2026. Accessed: 2026-04-22.

[2] GeeksforGeeks, "Multiagent planning in ai." https://www.geeksforgeeks.org/artificial-intelligence/multiagent-planning-in-ai/, 2026. Accessed: 2026-04-22.

[3] C. Richardson, "Microservices.io: Microservice architecture." https://microservices.io/, 2026. Accessed: 2026-04-25.

[4] Microsoft, "Microservices architecture style." https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices, 2024. Accessed: 2026-04-22.

[5] P. Ranjan, "Bdd automation framework using cucumber." https://community.ibm.com/community/user/blogs/pranjal-ranjan/2024/11/25/bdd-automation-framework-using-cucumber, 2024. Consultado el 22 de abril de 2026.

[6] IBM, "What is end-to-end testing?." https://www.ibm.com/think/topics/end-to-end-testing, 2024. Consultado el 22 de abril de 2026.

[7] Red Hat, "What are large language models?." https://www.redhat.com/en/topics/ai/what-are-large-language-models, 2026. Accessed: 2026-04-25.

[8] Zijie J. Wang and The Polo Club of Data Science, "Transformer explainer: Interactive visualization of transformers." https://poloclub.github.io/transformer-explainer/, 2024. Accessed: 2026-04-25.

[9] IBM, "What are llm parameters?." https://www.ibm.com/think/topics/llm-parameters, 2026. Accessed: 2026-04-22.

[10] DAIR.AI, "Prompt engineering guide." https://www.promptingguide.ai/, 2026. Consulté le 29 avril 2026.

[11] DAIR.AI, "Zero-shot prompting." https://www.promptingguide.ai/techniques/zeroshot, 2026. Consulté le 29 avril 2026.

[12] DAIR.AI, "Few-shot prompting." https://www.promptingguide.ai/techniques/fewshot, 2026. Consulté le 29 avril 2026.

[13] Portkey AI, "What is co-star prompt engineering framework?." https://portkey.ai/blog/what-is-costar-prompt-engineering/, 2024. Consulté le 29 avril 2026.

[14] Red Hat, "What is retrieval-augmented generation (rag)?." https://www.redhat.com/en/topics/ai/what-is-retrieval-augmented-generation, April 2026. Consulté le 25 avril 2026.

[15] Google Cloud, "What is a multi-agent system?." https://cloud.google.com/discover/what-is-a-multi-agent-system, 2026. Accessed: 2026-04-22.

[16] Conviva, "What is multi-agent environments?." https://www.conviva.ai/glossary/multi-agent-environments/, 2026. Consulté le 29 avril 2026.

[17] R. Karpurapu *et al.*, "Comprehensive evaluation and insights into the use of large language models in the automation of behavior-driven development acceptance test formulation." https://arxiv.org/abs/2403.14965, 2024.

[18] S. Santos *et al.*, "Increasing test coverage by automating bdd tests in proofs of concepts (pocs) using llm," in *Proceedings of the XXIII Brazilian Symposium on Software Quality (SBQS)*, SBC, 2024.

[19] I. Stojanović *et al.*, "Unit test generation multi-agent ai system for enhancing software documentation and code coverage." https://ieeexplore.ieee.org/document/10819096, 2024.

[20] Microsoft, "Autogen: A framework for building ai agents and applications." https://microsoft.github.io/autogen/stable/index.html, 2026. Consulté le 26 avril 2026.

[21] Qim info, "Java 17 : les nouveautés." https://www.qiminfo.ch/java-17-les-nouveautes/, 2026. Consulté le 26 avril 2026.

[22] IBM, "Java spring boot." https://www.ibm.com/think/topics/java-spring-boot, 2026. Consulté le 26 avril 2026.

[23] Red Hat, "What is rest architecture?." https://www.redhat.com/en/blog/rest-architecture, 2026. Consulté le 26 avril 2026.

[24] MySQL, "Mysql." https://www.mysql.com/, 2026. Consulté le 26 avril 2026.

[25] MySQL, "Mysql workbench." https://www.mysql.com/products/workbench/, 2026. Consulté le 26 avril 2026.

[26] Spring Boot, "What is rest architecture?." https://spring.io/projects/spring-data-jpa, 2026. Consulté le 26 avril 2026.

[27] Spring, "Spring cloud openfeign." https://spring.io/projects/spring-cloud-openfeign, 2026. Consulté le 26 avril 2026.

[28] SmartBear Software, "Swagger / openapi specification." https://swagger.io/specification/, 2024. Accessed: 2026-04-22.

[29] Swagger, "Open api." https://swagger.io/resources/open-api/, 2026. Consulté le 26 avril 2026.

[30] LangChain, "Langgraph documentation." https://langchain-ai.github.io/langgraph/, 2024. Accessed: 2026-04-22.

[31] LangChain, "Langchain documentation." https://python.langchain.com/docs/introduction/, 2024. Accessed: 2026-04-22.

[32] Groq, "Groq ai inference platform." https://groq.com/, 2026. Consulté le 27 avril 2026.

[33] Chroma, "Chromadb documentation." https://docs.trychroma.com/, 2024. Accessed: 2026-04-22.

[34] Cucumber, "Cucumber documentation." https://cucumber.io/docs/cucumber/, 2024. Accessed: 2026-04-22.

[35] JUnit Team, "Junit 5 user guide." https://junit.org/junit5/docs/current/user-guide/, 2024. Accessed: 2026-04-22.

[36] REST Assured, "Rest assured documentation." https://rest-assured.io/, 2024. Accessed: 2026-04-22.

[37] Apache Software Foundation, "Apache maven documentation." https://maven.apache.org/, 2024. Accessed: 2026-04-22.

[38] Mountainminds GmbH & Co. KG, "Jacoco java code coverage library." https://www.eclemma.org/jacoco/, 2025. Accessed: 2026-04-25.

[39] Pydantic, "Pydantic documentation." https://docs.pydantic.dev/, 2024. Accessed: 2026-04-22.

[40] PyYAML Team, "Pyyaml: The yaml framework for python." https://pyyaml.org/, 2026. Accessed: 2026-04-25.

[41] N. Reimers and I. Gurevych, "sentence-transformers/all-minilm-l6-v2." https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2, 2026. HuggingFace Model Card, consulté le 30 avril 2026.